# ToolUniverse: An open platform for democratizing AI scientists

Shanghua Gao[1], Richard Zhu[1,2,*], Pengwei Sui[1,*], Zhenglun Kong[1,*], Sufian Aldogom[1,*], Yepeng Huang[1], Ayush Noori[1], Reza Shamji[1,2], Krishna Parvataneni[3], Theodoros Tsiligkaridis[4], Marinka Zitnik[1,5,6,7,‡]

[1]Department of Biomedical Informatics, Harvard Medical School, Boston, MA
[2]Harvard College, Harvard University, Cambridge, MA
[3]Massachusetts Institute of Technology, Cambridge, MA
[4]MIT Lincoln Laboratory, Lexington, MA
[5]Kempner Institute for the Study of Natural and Artificial Intelligence, Harvard University, Cambridge, MA
[6]Broad Institute of MIT and Harvard, Cambridge, MA
[7]Harvard Data Science Initiative, Cambridge, MA
[*]Co-second authors; ‡Correspondence: marinka@hms.harvard.edu

arXiv:2509.23426v3 [cs.AI] 7 Aug 2026

AI scientists are AI agents powered by conversational large language models that carry out complex analyses by calling scientific tools, querying datasets, and interacting with experimental systems, other agents, and human researchers [1]. Despite rapid progress, most are bespoke and difficult to reuse because they lack standardized interfaces for tools and data [2, 3]. Scientific research requires that AI scientists act on biological and physical systems rather than reason through text alone, operating within an environment of tools, agentic skills, replayable agentic memory, and new agentic capabilities introduced in the fast advancing field [1, 4]. The tools, skills, and capabilities supporting this interaction vary in purpose and complexity, require distinct inputs and parameters, depend on specific runtime environments, and run locally or through application programming interfaces (APIs) and laboratory systems. The tools among them span modalities from protein and nucleic acid sequences and molecular structures to microscopy images and biological networks, and their overlapping functionality complicates selecting and executing an appropriate tool for an objective expressed in natural language.

We present TOOLUNIVERSE, a platform for building AI scientists across open- and closed-weight language models and agents. TOOLUNIVERSE implements scientific tool use through six modules, Tool Manager, Tool Finder, Tool Caller, Tool Composer, Tool Discoverer, and Tool Optimizer, that together enable AI scientists to discover, retrieve, execute, and compose tools from a library of more than 2,700 scientific tools as of July 2026 (Figure 1a; Supplementary Note S1 and S5). TOOLUNIVERSE is available at https://aiscientist.tools and accessible through a Python API, HTTP, the Model Context Protocol (MCP), and direct function calls (Supplementary Note S4).

TOOLUNIVERSE defines a standard for AI-tool interaction that each tool implements so that AI scientists can call it. A tool specifies its intended use in natural language, a schema naming and typing all inputs and outputs with their constraints, and a backend-agnostic invocation format, so the same call invoked in different contexts can execute a Python library, a machine learning model, a dataset endpoint, or a laboratory instrument without tool-specific configuration (Supplementary Note S3). The composition of the library by tool type and its coverage by data modality and

application area are summarized in Supplementary Tables S2 and S3. The library's tools are also accessible through over 130 research skills (Supplementary Table S4).

TOOLUNIVERSE manages tools through six components that cover how they are registered, found, called, composed, created, and refined (Figure 1b–g; Supplementary Note S3). Tool Manager registers local functions and remote services behind a single interface, so tools with different backends are invoked the same way; Tool Finder retrieves the most relevant tools for a natural language request by combining keyword, language model, and embedding based search; and Tool Caller executes the selected tool with validated inputs, returning structured results or actionable errors through Python, the TOOLUNIVERSE API, MCP, and HTTP. Tool Composer chains tools into sequential, parallel, or feedback driven workflows; Tool Discoverer creates new runnable tools from natural language descriptions and automatically validates them, with human review required before registration; and Tool Optimizer refines tool descriptions to match observed behavior. We benchmark Tool Finder across its keyword, embedding, and language model search strategies, and evaluate the embedding strategy under a range of interchangeable open and hosted encoders; guided by these results, we add strong performing encoders as built-in, selectable options. We also benchmark TOOLUNIVERSE on standard scientific tasks, where an agent or language model equipped with TOOLUNIVERSE outperforms both the same agent without it and a purpose-built biomedical agent [5] (Supplementary Note S8).

Language models and agents invoke TOOLUNIVERSE in context through tool specifications or through agentic frameworks such as Gemini CLI and Claude Code; end-to-end examples are in Supplementary Note S4. Building on these interfaces, TOOLUNIVERSE can be used in two complementary ways. Researchers developing new models can leverage its integrated environment of tools and agentic skills to teach models tool use through reinforcement learning, where agents acquire policies for selecting and parameterizing tools and improve over long agentic runs, as in ATHENA [6]. Existing agents and language models can integrate TOOLUNIVERSE to expand their available tools through standardized retrieval and invocation interfaces, as in Virtual Lab [3], Medea [7], and GeneAgent [8].

Beyond individual tools, TOOLUNIVERSE's research skills are natural language workflows that orchestrate multiple tools into complex analyses. A biologist describes a goal in plain language, for example performing differential expression analysis of RNA-seq data, assessing a candidate drug target, or reviewing disease evidence, and the matching skill runs the end-to-end workflow. These skills span genomics, transcriptomics, proteomics, variant interpretation, drug discovery, and single cell and spatial biology (Supplementary Note S6), with examples provided in Supplementary Note S9 for cancer target assessment, chemical compound analysis against a druggable target, and clinical trial analysis. TOOLUNIVERSE also verifies, logs, and governs tools across their lifecycle, confirming through test cases and review of input-output traces that each tool functions as described before use, recording tool calls and arguments through optional logging hooks for inspection and audit, and applying additional governance safeguards to mitigate

safety risks [9] (Supplementary Note S7).

We envision TOOLUNIVERSE growing as an open platform. Because every tool implements the same AI-tool interaction standard, TOOLUNIVERSE scales without changes to the core platform, whether contributors add local tools, register remote tools over MCP, or generate them with Tool Discoverer (Supplementary Note S3). Community-contributed tools and skills are incorporated only if they meet explicit criteria, namely conformance to the TOOLUNIVERSE protocol, test cases covering typical and edge conditions, human expert review of input-output traces, and tool quality thresholds (Supplementary Note S7). As human-AI co-science evolves further, reproducibility and safety grow in importance. TOOLUNIVERSE supports reproducibility through versioned open-source tools and audit logging, and safety through human-in-the-loop oversight of consequential actions [9].

**Data availability.** A complete, versioned list of the sources accessed by each tool is provided with the code at https://github.com/mims-harvard/ToolUniverse. Supplementary Information reports case studies and benchmarking obtained from live tool executions. All data supporting the findings of this study are available within the paper and its Supplementary Information and from the public repositories and web services accessed by its tools. TOOLUNIVERSE is available at https://aiscientist.tools, with documentation at https://github.com/mims-harvard/ToolUniverse and additional information at https://zitniklab.hms.harvard.edu/ToolUniverse.

**Code availability.** TOOLUNIVERSE source code is available under Apache License 2.0 at https://github.com/mims-harvard/ToolUniverse. Development follows semantic versioning with tagged, numbered releases, and the version described in this manuscript is v1.3.1. A citable archive of this release is deposited on Zenodo [10]. TOOLUNIVERSE can be installed from the Python Package Index with `pip install tooluniverse` (https://pypi.org/project/tooluniverse), and the TOOLUNIVERSE Model Context Protocol (MCP) connector is at https://aiscientist.tools/mcp. Documentation, tutorials, and contribution guidelines are provided in the repository.

**Acknowledgments.** We thank Nicholas Yang and Yuchang Su for their contributions to TOOLUNIVERSE. We gratefully acknowledge the support of NSF CAREER 2339524, ARPA-H Biomedical Data Fabric (BDF) Toolbox Program, Harvard Data Science Initiative, Amazon Faculty Research, Google Research Scholar Program, AstraZeneca Research, Roche Alliance with Distinguished Scientists (ROADS) Program, Sanofi iDEA-iTECH Award, GlaxoSmithKline Award, Boehringer Ingelheim Award, Merck Award, Optum AI Research Collaboration Award, Pfizer Research, Gates Foundation (INV-079038), Chan Zuckerberg Initiative, John and Virginia Kaneb Fellowship at Harvard Medical School, Biswas Computational Biology Initiative in partnership with the Milken Institute, Harvard Medical School Dean's Innovation Fund for the Use of Artificial Intelligence, and the Kempner Institute for the Study of Natural and Artificial Intelligence at Harvard University. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the funders.

**Competing interests.** None.

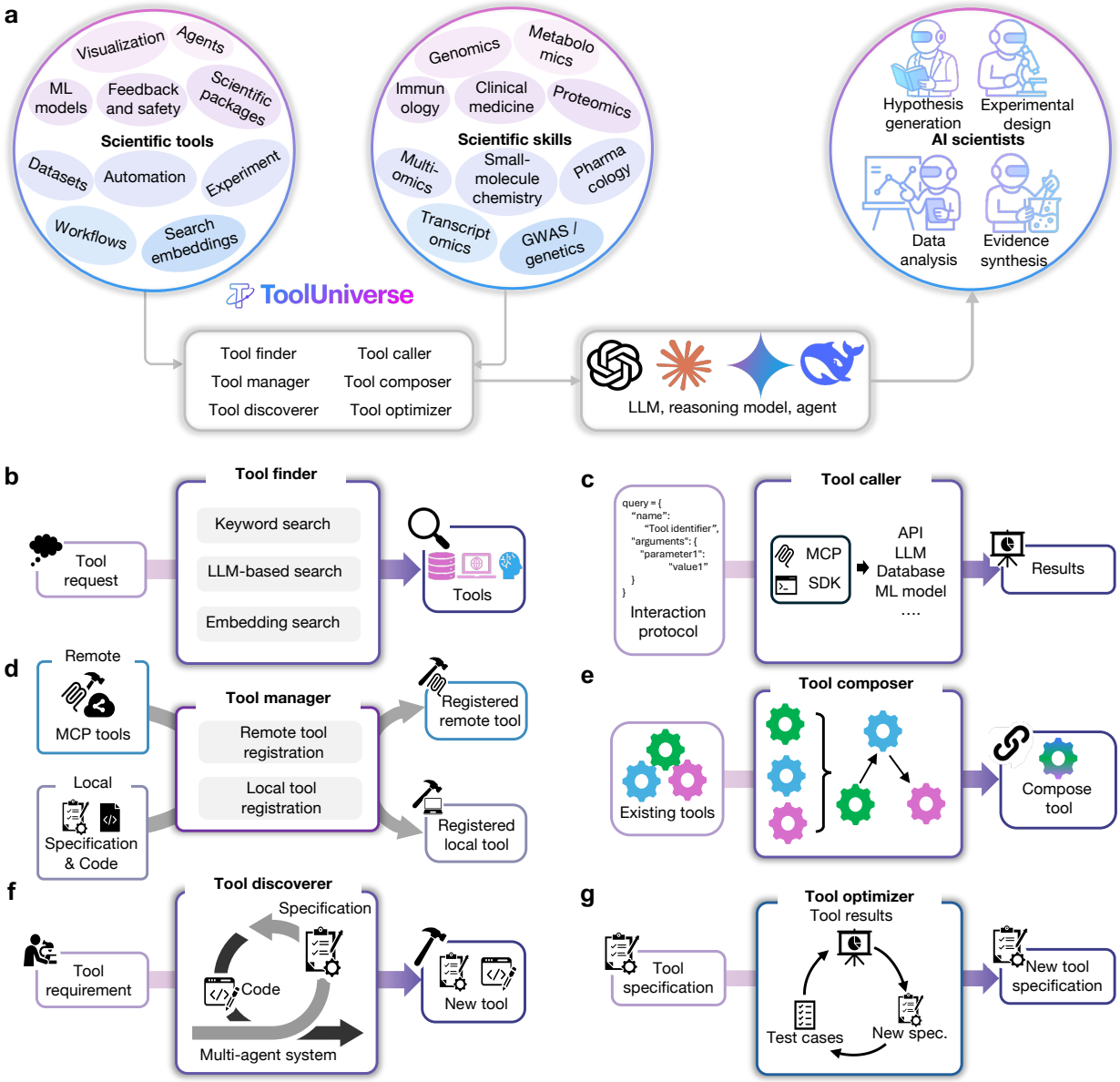

a
Visualization
Agents
ML models
Feedback and safety
Scientific packages
Scientific tools
Datasets
Automation
Experiment
Workflows
Search embeddings
Genomics
Metabolomics
Immunology
Clinical medicine
Proteomics
Scientific skills
Multi-omics
Small-molecule chemistry
Pharmacology
Transcriptomics
GWAS / genetics
Hypothesis generation
Experimental design
AI scientists
Data analysis
Evidence synthesis
ToolUniverse
Tool finder
Tool caller
Tool manager
Tool composer
Tool discoverer
Tool optimizer
LLM, reasoning model, agent
b
Tool finder
Tool request
Keyword search
LLM-based search
Embedding search
Tools
c
Tool caller
query = {
"name":
"Tool identifier",
"arguments": {
"parameter1":
"value1"
}
}
Interaction protocol
MCP
SDK
API
LLM
Database
ML model
....
Results
d
Remote
MCP tools
Tool manager
Remote tool registration
Local tool registration
Registered remote tool
Local
Specification & Code
Registered local tool
e
Tool composer
Existing tools
Compose tool
f
Tool discoverer
Tool requirement
Specification
Code
Multi-agent system
New tool
g
Tool optimizer
Tool results
Tool specification
Test cases
New spec.
New tool specification

**Figure 1**

**Figure 1: a.** TOOLUNIVERSE is an open platform for building AI scientists that integrates 2,700 scientific tools and 130 research skills as of July 2026. The tools span ML models, agents, datasets, workflows, scientific packages, laboratory automation and experimentation, visualization, search embeddings, and human feedback and safety (left; full taxonomy in Supplementary Table S1), and are exposed through the research skills grouped by scientific area, including genomics, transcriptomics, proteomics, pharmacology, small molecule chemistry, clinical medicine, immunology, metabolomics, multi-omics, and GWAS and genetics (middle; Supplementary Table S4). Six modules (Tool Finder, Tool Caller, Tool Manager, Tool Composer, Tool Discoverer, and Tool Optimizer; detailed in panels b–g) connect the platform, through the AI-tool interaction standard, to large language models, reasoning models, and agents (GPT, Claude, Gemini, and DeepSeek). Connected to TOOLUNIVERSE, these models act as AI scientists that perform research tasks, including hypothesis generation, experimental design, data analysis, and evidence synthesis (right). End-to-end examples of building AI scientists on TOOLUNIVERSE are provided in Supplementary Note S4, and a quantitative evaluation of the platform in Supplementary Notes S8 and S9. **b.** Tool Finder identifies relevant tools using keyword search, LLM-based in-context search, and embedding-based similarity search. **c.** Tool Caller validates inputs, dynamically loads tools, dispatches calls through TOOLUNIVERSE.run() or MCP, and returns tool outputs. **d.** Tool Manager integrates local and remote tools, registering local tools via JSON descriptions and decorators and adding remote tools through MCP for privacy- or dependency-constrained setups. **e.** Tool Composer chains multiple tools into composite workflows, supporting sequential, parallel, and feedback-driven orchestration of heterogeneous tools. **f.** Tool Discoverer is a multi-agent system that generates new tools from natural-language requirements, performing definition synthesis, automated code generation, validation, and iterative refinement to produce runnable tools. **g.** Tool Optimizer is a multi-agent system that iteratively refines tool descriptions to improve clarity, accuracy, and usability, combining test generation, execution analysis, and feedback-driven refinement. ML, Machine Learning; LLM, Large Language Model; MCP, Model Context Protocol; API, Application Programming Interface; SDK, Software Development Kit.

## References


1. Gao, S. *et al.* Empowering biomedical discovery with AI agents. *Cell* **187**, 6125–6151 (2024).

2. Boiko, D. A., MacKnight, R., Kline, B. & Gomes, G. Autonomous chemical research with large language models. *Nature* **624**, 570–578 (2023).

3. Swanson, K., Wu, W., Bulaong, N. L., Pak, J. E. & Zou, J. The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies. *Nature* **646**, 716–723 (2025).

4. Wang, H. *et al.* Scientific discovery in the age of artificial intelligence. *Nature* **620**, 47–60 (2023).

5. Huang, K. *et al.* Autonomous biomedical research with an artificial intelligence agent. *Science* eadz4351 (2026).

6. Gao, S. *et al.* An AI agent for treatment reasoning over a biomedical tool universe. *arXiv:2606.28692* (2026).

7. Sui, P. *et al.* Medea: An AI agent for therapeutic reasoning across biological contexts. *bioRxiv* (2026).

8. Wang, Z. *et al.* GeneAgent: self-verification language agent for gene-set analysis using domain databases. *Nature Methods* **22**, 1677–1685 (2025).

9. Tang, X. *et al.* Risks of AI scientists: prioritizing safeguarding over autonomy. *Nature Communications* **16**, 8317 (2025).

10. ToolUniverse: An open platform for democratizing AI scientists (v1.3.1). https://doi.org/10.5281/zenodo.21121729 (2026).

11. Gao, S. *et al.* TxAgent: An AI agent for therapeutic reasoning across a universe of tools (2025). 2503.10970.

12. Wang, H., He, Y., Coelho, P. P., Bucci, M. *et al.* SpatialAgent: An autonomous AI agent for spatial biology. *bioRxiv* (2025).

13. Open Targets. 24.09 platform release now live! https://community.opentargets.org/t/24-09-platform-release-now-live/1556?ref=blog.opentargets.org (2024).

14. Pearce, J. D. *et al.* TranscriptFormer: A generative cell atlas across 1.5 billion years of evolution. *Science* **393**, aec8514 (2026).

15. Li, M. M. *et al.* Contextual AI models for single-cell protein biology. *Nature Methods* **21**, 1546–1557 (2024).

16. Broad DepMap. DepMap 24Q2 Public. *Figshare* (2024).

17. Swanson, K. *et al.* ADMET-AI: a machine learning ADMET platform for evaluation of large-scale chemical libraries. *Bioinformatics* **40**, btae416 (2024).

18. Passaro, S. *et al.* Boltz-2: Towards accurate and efficient binding affinity prediction. *bioRxiv* (2025).

19. Himes, B. E. *et al.* RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene that modulates cytokine function in airway smooth muscle cells. *PLoS One* **9**, e99625 (2014).

20. Laurent, J. M. *et al.* LAB-Bench: Measuring capabilities of language models for biology research. *arXiv preprint arXiv:2407.10362* (2024).

21. Jumper, J. *et al.* Highly accurate protein structure prediction with AlphaFold. *Nature* **596**, 583–589 (2021).

22. Zdrazil, B. *et al.* The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research* **52**, D1180–D1192 (2024).

Supplementary Information for

# ToolUniverse: An open platform for democratizing AI scientists

Shanghua Gao[1], Richard Zhu[1,2,*], Pengwei Sui[1,*], Zhenglun Kong[1,*], Sufian Aldogom[1,*], Yepeng Huang[1], Ayush Noori[1], Reza Shamji[1,2], Krishna Parvataneni[3], Theodoros Tsiligkaridis[4], Marinka Zitnik[1,5,6,7,‡]

[1]Department of Biomedical Informatics, Harvard Medical School, Boston, MA
[2]Harvard College, Harvard University, Cambridge, MA
[3]Massachusetts Institute of Technology, Cambridge, MA
[4]MIT Lincoln Laboratory, Lexington, MA
[5]Kempner Institute for the Study of Natural and Artificial Intelligence, Harvard University, Cambridge, MA
[6]Broad Institute of MIT and Harvard, Cambridge, MA
[7]Harvard Data Science Initiative, Cambridge, MA
[*]Co-second authors; ‡Correspondence: marinka@hms.harvard.edu

TOOLUNIVERSE website is at https://aiscientist.tools
TOOLUNIVERSE code is at https://github.com/mims-harvard/ToolUniverse
TOOLUNIVERSE package is at https://pypi.org/project/tooluniverse

This PDF file includes:

Supplementary Notes 1 to 9

Supplementary Figures 1 to 9

Supplementary Tables 1 to 4

## S1 Supplementary Note: Overview of TOOLUNIVERSE

TOOLUNIVERSE is an ecosystem that exposes tools to any open or closed AI model, *i.e.*, large language model (LLM), agent, or large reasoning model (LRM), allowing users to build AI research assistants (*i.e.*, AI scientists) without additional training or finetuning. To achieve that, TOOLUNIVERSE provides the user-specified LLM/LRM/agent with a scientific toolkit (Supplementary Table S1). While advanced models possess planning and reasoning capabilities, scientific research cannot be conducted through reasoning alone. TOOLUNIVERSE addresses this by exposing executable tools that return experimental data, database query results, and model predictions. TOOLUNIVERSE defines an AI-tool interaction standard that makes tools understandable to LLMs, LRMs, and AI agents through natural-language descriptions and typed schemas, abstracting away backend differences.

TOOLUNIVERSE is extensible and allows tools to be easily added, optimized, or created. It hosts a toolkit of over 2,700 scientific tools and is designed for integration with language models, agents, and reasoning models. Current tools in TOOLUNIVERSE include: foundation models, fine-tuned LLMs, LRMs, and other ML models exposed as callable endpoints; agentic planners and tool routers; domain libraries and simulators; and systems for human-in-the-loop feedback and lab automation with instrument control. TOOLUNIVERSE also provides data and retrieval utilities, such as data sources, knowledge bases, vector search with embedding generators, and complete retrieval-augmented generation (RAG) pipelines. For integration and governance, the ecosystem offers external service connectors, typed API clients, privacy guardrails, safety checklists, compliance controls, and audit logs. TOOLUNIVERSE supports high-level scientific and operational workflows through visualization dashboards, experiment design tools with ELN/LIMS integration, and workflow engines and orchestration schedulers. Despite the backend heterogeneity of these tools, which span machine learning models, AI agents, software utilities, robotics, databases, and APIs, all are presented to the AI scientist through the AI-tool interaction standard, which we describe next.

**AI-tool Interaction Standard.** TOOLUNIVERSE defines an AI-tool interaction standard that makes the backend agnostic to users and simplifies the addition of new tools. Each tool includes three elements: (i) a natural-language description of the tool's purpose and expected behavior, (ii) a typed schema listing every input parameter and output field with names, types, and constraints, and (iii) a uniform invocation format that abstracts away backend differences. By supplying this information within an AI model's context window, TOOLUNIVERSE enables any LLM, reasoning model, or agent to understand and invoke tools without tool-specific configuration.

**Core Components.** This standard supports six core modules. Tool Manager registers local functions and remote services, Tool Finder searches for options from over 2,700 candidates based on user requirements, and Tool Caller executes tools with validated arguments and structured returns. For complex tasks, Tool Composer assembles multiple tools into workflows. Tool Discoverer generates new tools from natural language descriptions, and Tool Optimizer refines tool descriptions through iterative testing and feedback. These components and the AI-tool interaction standard enable creating AI scientists: specialized AI models that combine reasoning with access to tools to perform multi-step research tasks.

## S2 Supplementary Note: AI-tool Interaction Standard in TOOLUNIVERSE

TOOLUNIVERSE is designed to support an ecosystem of tools with a wide range of abilities. This note specifies the AI-tool interaction standard through which every one of them is managed and presented to the client in the same form.

### S2.1 Tool Description Schema

Supplementary Figure S3 shows the tool description schema in TOOLUNIVERSE. The AI-tool interaction standard exposes every tool via a description containing its name; a functional description; a list of parameters, where each parameter is explicitly defined with its own name, description, data type, and required status; and a return schema that shows the data structure of the returned data. An example tool description is shown in Supplementary Figure S4. This description is provided to clients such as LLMs, reasoning models, AI agents, and human users to help them understand how to use the tool. For instance, when the client is an LLM, the tool description is supplied within its context window, thereby granting it the necessary information to interact with tools from TOOLUNIVERSE.

### S2.2 Tool Interaction Schema

TOOLUNIVERSE processes requests through a standard interaction schema, illustrated in Supplementary Figure S5. All interactions are formatted as a single string that encodes a function call, specifying the desired tool's name and its input arguments. This schema provides the foundation for all interactions with TOOLUNIVERSE, enabling tool search, calling, discovery, optimization, and composition.

### S2.3 Accessing Tools from TOOLUNIVERSE

Using the AI-tool interaction standard, TOOLUNIVERSE provides an interface for human users and AI agents to access tools. The tool can be invoked by executing:

```
tooluniverse.run(tool_call_schema)
```

where `tool_call_schema` is a dictionary following the interaction schema: {'name': name of the tool, 'arguments': parameters required by the tool}. This approach abstracts away backend complexity. Regardless of a tool's underlying implementation, it is presented to the client (the user of the tool, such as LLMs, reasoning models, AI agents) through the AI-tool interaction standard. To use a tool, the client consults the tool description to construct a request that adheres to the interaction schema. This request is then sent to TOOLUNIVERSE via either a local interface or a remote http connection. Tool Caller processes the request, executes the specified tool, and returns the results to the client. This eliminates the need for complex configurations, regardless of backend or runtime differences. For example, users can query a new database without writing database-specific SQL, run machine learning models without configuring GPUs or environments, or access web-based lab equipment through a single, standardized tool call request.

## S3 Supplementary Note: Core Modules of TOOLUNIVERSE

TOOLUNIVERSE organizes the ecosystem into six modules for tool management. Tool Manager integrates tools into TOOLUNIVERSE through local and remote registration. Tool Finder searches

for suitable options from over 2,700 candidates based on user requirements. Tool Caller executes tools with validated arguments and structured returns. For complex tasks, Tool Composer assembles multiple tools into workflows. Tool Discoverer generates new tools from natural language descriptions. Tool Optimizer refines tool descriptions through iterative testing and feedback.

## S3.1 Tool Manager

Tool Manager is designed to simplify the process of adding new tools to TOOLUNIVERSE through two registration modes. For local tools, which require no special dependencies, only a JSON description (including name, descriptions, typed arguments, and configurations) and a corresponding callable function are needed. Once registered, any connected AI model can invoke them. For remote tools, which may have specialized dependencies or restricted access, TOOLUNIVERSE integrates them through MCP by registering the server address, which loads all hosted tools into TOOLUNIVERSE's tool list. After registration, both local and remote tools expose the same AI-tool interaction standard, making backend differences invisible to AI models.

**Local Tool Registration.** For local tool registration, to add a tool to TOOLUNIVERSE, both a tool configuration and a corresponding tool class are required. The tool configuration is a dictionary that specifies the tool according to the AI-tool interaction standard and includes the necessary settings for its execution. The tool class defines an initialization function that sets up the tool based on its description, as well as a run function that processes tool call arguments in accordance with the interaction schema. Local tool registration within the Tool Manager is facilitated through an easy-to-use decorator function, `register_tool(Class_name, tool_config)`, which decorates the tool class as illustrated in Supplementary Figure S7. Here, `Class_name` is the string name of the tool class, and `tool_config` is the configuration containing both the tool description and required settings. Once registered, the tool is automatically integrated into TOOLUNIVERSE without further manual configuration. This registration process allows users to incorporate custom tools into TOOLUNIVERSE, enabling coordination with other existing tools and empowering the creation of customized AI scientists.

**Remote Tool Registration.** Remote Tool Registration enables the integration of tools that are private, require specialized configurations, or operate within restricted environments, and therefore cannot be made publicly available. Once registered remotely, these tools are added to TOOLUNIVERSE and can be accessed and executed in the same manner as standard tools.

To achieve this, the Tool Manager includes an MCP Auto Loader Tool that accepts the address of an MCP server and registers all of its tools into the TOOLUNIVERSE's tool list. After the MCP Auto Loader Tool has loaded the remote tools, they are integrated into the TOOLUNIVERSE with the same functionality as other tools. For the remote side, TOOLUNIVERSE supports two methods for setting up a remote tool. The first is building standard tools that support MCP. To further simplify the process and make remote tool registration identical to local registration, the Tool Manager also provides a decorator function, `register_remote_tool(Class_name, tool_config, mcp_config)`, which decorates the tool class as illustrated in Supplementary Figure S8. In this function, the `Class_name` and `tool_config` parameters are the same as those used in `register_tool(Class_name, tool_config)`, while `mcp_config` defines the configuration for the MCP server, such as the host address and port used for the service.

## S3.2 Tool Finder

The Tool Finder retrieves tools from a library of more than 2,700 scientific tools using three strategies, each trading off speed, semantic understanding, and compute cost. The input is a natural language query from the client, describing the task they wish to achieve or the specific capabilities required from the tools. The three strategies are: keyword search using TF-IDF scoring, LLM-based in-context search that reasons about multi-step or abstract requests, and embedding search that encodes queries and tool descriptions using a fine-tuned GTE-Qwen2-1.5B model.

**Keyword search.** Keyword search parses a user's query through a multi-stage workflow involving tokenization via regular expressions, the removal of over 45 common English stop words, and suffix-based stemming using 20 morphological rules to reduce words to their root form. To capture multi-word concepts, this method also generates n-grams (bigrams and trigrams). These processed keywords and phrases are then matched against a pre-built index of tool descriptions, which has undergone the same processing. Relevance is scored using a term frequency-inverse document frequency (TF-IDF) algorithm, calculated as: $\text{Relevance} = \text{TF} \times \text{IDF} \times \log(1 + \text{QueryFrequency})$, where TF measures how often a term appears in a document, IDF reflects how unique the term is across all documents, and QueryFrequency indicates how often the term appears in the user's query. The scoring model adjusts relevance scoring by applying a hierarchical bonus structure to the relevance score. Matches found in a tool's name receive the largest weight with a 2.0× bonus multiplier, followed by a 1.5× multiplier for exact phrase matches within descriptions. This keyword search approach provides a keyword-based search solution that operates independently of machine learning models and can be used across different resource levels.

**LLM in-context search.** The LLM in-context search uses the reasoning capabilities of a Large Language Model to interpret user intent more holistically. Rather than relying on simple keyword matching, a detailed prompt for tool selection is constructed. This prompt contextualizes the user's task description with the tool descriptions of a candidate set of tools in TOOLUNIVERSE. The LLM is then tasked with analyzing this rich context to infer a suitable tool or sequence of tools required to fulfill the user's request. This method is used for interpreting complex, multi-step, or abstract queries that demand logical inference. While its application can be constrained by the finite context window of the model, it offers flexibility in understanding abstract goals. The LLM in-context search is powered by the agentic tool implementation in TOOLUNIVERSE. This allows the user to provide a configuration file that defines the prompt and tool descriptions, without needing to manage the backend LLM inference processes.

**Embedding search.** Embedding search is a vector-based method that retrieves tools by matching the semantic similarity between a user's query and the tool's description. To achieve this, we fine-tune the GTE-Qwen2-1.5B language embedding model using pairs of synthetic user queries and augmented tool descriptions, training it to understand the connection between a user's intent and a tool's function. The process involves two stages. First, in an offline indexing stage, each tool's description is passed through the embedding model to generate a semantic vector that captures its meaning. These vectors are then stored and indexed in a specialized vector database. Later, during the online querying stage, a user's natural language query is converted into a query vector using the same model. Finally, relevant tools are discovered by calculating the cosine similarity between the user's query vector and all the tool vectors in the database, identifying the closest matches.

## S3.3 Tool Caller

The Tool Caller is the primary execution engine in TOOLUNIVERSE and is accessible through four interfaces: direct Python function calls, the TOOLUNIVERSE Python API, MCP, and HTTP API, making tools accessible from any programming environment or remote client. Upon initialization, the Tool Caller is configured with a manifest of available tools, including their descriptions and settings. To mitigate the significant system overhead associated with loading all tools simultaneously, it employs a dynamic loading strategy. A specific tool is loaded into memory only upon its first request and is then cached for a duration to efficiently handle subsequent calls. During this loading process, the Tool Caller injects the necessary configurations, such as API endpoints and authentication keys, into the corresponding tool class.

When a tool execution request is received, the Tool Caller first parses it to extract the tool name and arguments. It then performs a rigorous validation check, ensuring the provided arguments conform to the data types and structural requirements defined with the AI-tool interaction standard. Once validated, the Tool Caller dispatches the arguments to the tool's primary execution method, such as `run()`, as illustrated in Supplementary Figure S6. The resulting output is then returned to the client through TOOLUNIVERSE's communication protocols. If any step in this process fails, from loading to validation or execution, the system generates and returns a descriptive error message. This feedback mechanism helps the client diagnose the issue and revise the request accordingly.

## S3.4 Tool Composer

For complex tasks, users can create new composite tools by combining existing ones. Tool Composer chains tools with heterogeneous backends into multi-step workflows. Using the Tool Caller for direct in-code execution, Tool Composer generates a container function that exposes both the Tool Caller and TOOLUNIVERSE as in-line, executable primitives. The container function, implemented as `compose(arguments, tooluniverse, call_tool)`, serves as the execution backbone for Tool Composer. It contains the logic for coordinating different types of tools so they work together in a single workflow. The `arguments` parameter specifies the tool call arguments that follow the interaction schema of TOOLUNIVERSE, the `tooluniverse` is an instance of TOOLUNIVERSE that provides all available functions that TOOLUNIVERSE can support, and the `call_tool` parameter is a callable interface of Tool Caller that abstracts the invocation of individual tools in TOOLUNIVERSE. By integrating these components, the container function enables three execution patterns: (i) chaining the output of one tool into the input of the next, (ii) broadcasting a single query across multiple tools in parallel, and (iii) constructing agentic loops in which a reasoning model generates tool calls, executes them, and incorporates the feedback for adaptive multi-step analysis. As illustrated in Supplementary Figure S9, a composed tool can run several literature search tools concurrently and then invoke a summarization agent to synthesize the findings, demonstrating heterogeneous workflow construction in which each step is driven by tool execution.

## S3.5 Tool Discoverer

Tool Discoverer automatically generates new tools, including both tool definitions and executable code, from high-level natural language descriptions. Leveraging the workflow composition capabilities of TOOLUNIVERSE via Tool Composer, we construct an agentic multi-stage workflow

that transforms a plain-text functional request into a runnable tool with limited human intervention. The process integrates tool discovery, structured definition generation, code implementation, and iterative quality refinement, ensuring that the final tool adheres to functional requirements and ecosystem conventions. Here, "limited human intervention" refers to the automated generation loop and does not bypass human review: every generated tool, like all tools in TOOLUNIVERSE, undergoes the human-in-the-loop expert review described in the Supplementary Note on Evaluation of Tools (Supplementary Note S7) before it is added to the platform.

Through iterative feedback loops combining search, analysis, and code optimization, the system progressively refines the generated tool until it reaches predefined quality criteria. The process terminates when target scores are achieved or iteration limits are reached, producing tools intended for downstream use.

**Core principles and architecture.** The Tool Discoverer system is built around four core principles: pattern-guided generation that reuses functional patterns and conventions from existing tools to ensure ecosystem consistency; structured definition synthesis that transforms unstructured requests into tool definitions; automated code generation and validation that produces executable implementations with integrated testing; and iterative refinement that uses feedback from analysis and testing to drive targeted improvements.

The system integrates three specialized components working in sequence. The SpecificationGenerator converts the natural language description and retrieved reference implementations into a structured definition, including tool name, description, parameter definitions, return schema, and category metadata. The ImplementationGenerator produces code that follows standard software engineering practices, includes complete dependency handling, integrates with the TOOLUNIVERSE registry, and implements error handling for failure cases. The QualityEvaluator assesses the generated tool across functionality, reliability, maintainability, performance, and test coverage, scoring each dimension from 0–10 and computing an overall weighted score.

**Generation process.** The generation workflow begins with the discovery stage, which accepts a tool description as inputs. It searches GitHub, PyPI, and existing TOOLUNIVERSE tools for reference implementations using multiple search strategies (semantic similarity and keyword-based search). Results are compiled into a final set of references. Next, the definition stage uses an agentic tool to produce a complete tool configuration. This includes tool name, descriptions, parameter definitions with type annotations and descriptions, JSON-schema-based return type definition, and other metadata. The output conforms to TOOLUNIVERSE schema requirements and naming conventions, enabling immediate downstream integration. The implementation phase employs template-driven code generation to produce runnable code, incorporating required imports, type-hinted function signatures, error handling, integration hooks via the @register_tool decorator in TOOLUNIVERSE, and a module structure suitable for integration. Finally, the system conducts multiple quality assessments, combining code analysis, dynamic testing with auto-generated test cases, and execution profiling. Each dimension is scored, with a target minimum of 9/10 for deployment-oriented settings. An iterative refinement loop applies targeted improvements until the threshold is met. Generated tools are packaged for immediate inclusion in TOOLUNIVERSE, producing a JSON configuration file with metadata, a source file with the complete implementation, and dependency information for reproducible installation.

## S3.6 Tool Optimizer

Tool Optimizer is designed to refine tool descriptions with respect to clarity, accuracy, and interpretability for models. Using Tool Composer, we build an agentic tool description optimization workflow where tool descriptions are optimized through an iterative multi-round process combining automated test case generation, real tool execution analysis, and feedback refinement. Through feedback-driven iterations, the system refines both tool descriptions and parameter descriptions, eliminating redundancy between them while checking alignment through empirical validation. The optimization process automatically terminates when quality thresholds are met or maximum iterations are reached, producing descriptions that aim to support tool use for AI agents.

**Core principles and tools.** The Tool Optimizer employs a multi-round iterative optimization strategy to automatically enhance tool documentation quality. The optimizer is built on three core principles: (1) test-driven optimization that validates descriptions against actual tool execution results, (2) multi-dimensional quality assessment across six standardized criteria, and (3) feedback-driven improvement that uses insights from previous optimization rounds to guide subsequent iterations. The optimizer implements a compositional architecture consisting of three specialized components working in concert. The TestCaseGenerator creates diverse test scenarios based on tool configurations and adaptively generates targeted test cases in later rounds using feedback from previous iterations. The DescriptionAnalyzer examines the alignment between existing descriptions and actual tool behavior by analyzing test execution results, then generates optimized descriptions for both tool-level and parameter-level documentation that better reflect the tool's true functionality while ensuring consistency with real usage patterns and eliminating redundancy between tool and parameter descriptions. Finally, the DescriptionQualityEvaluator provides objective scoring on a 0-10 scale across six quality dimensions: clarity, accuracy, completeness, conciseness, user-friendliness, and redundancy avoidance.

**Optimization process.** The optimization process begins with initial test case generation followed by tool execution to gather execution results. The system then enters an iterative optimization loop where each round generates enhanced test cases based on previous feedback, analyzes accumulated test results to propose improved descriptions, optimizes both tool and parameter descriptions, and evaluates quality against predefined thresholds. The process continues until either the satisfaction threshold is met (typically 8.0/10 for deployment-oriented settings) or the maximum iteration limit is reached (default: 3 rounds). This adaptive approach ensures comprehensive coverage while maintaining runtime constraints.

# S4 Supplementary Note: Building AI scientists with TOOLUNIVERSE

A customized AI scientist can be developed by integrating TOOLUNIVERSE with LLMs, reasoning models, and AI agents. In this configuration, the LLMs and reasoning models provide the core capabilities for reasoning and tool usage, while TOOLUNIVERSE serves as the scientific environment for interaction and experimentation. The development process involves three steps: (i) installing TOOLUNIVERSE with a single command (`pip install tooluniverse`), (ii) connecting TOOLUNIVERSE to the chosen model so it can access the tools provided by TOOLUNIVERSE, and (iii) instructing the model to use these tools to address a given scientific problem.

Once the setup is complete, the AI scientist operates as follows: given a user instruction or task, it formulates a plan or hypothesis, employs Tool Finder in TOOLUNIVERSE to identify

relevant tools, and iteratively applies these tools to gather information, conduct experiments, verify hypotheses, and request human feedback when necessary. For each required tool call, the AI scientist generates arguments that conform to the AI-tool interaction standard, after which Tool Caller executes the tool and returns the results for further reasoning.

The models used to construct AI scientists can include LLMs, reasoning models, and AI agents. LLMs may be API-based, such as Claude or GPT, or open-weight models, such as LLaMA, DeepSeek, or Qwen. Large reasoning models enhance problem-solving capabilities by applying built-in chains of thought to analyze the current step before interacting with TOOLUNIVERSE. Agentic systems, such as Gemini CLI or Claude Code, integrate reasoning models with agentic feedback loops to autonomously manage multi-step problem solving and tool use. In addition to general-purpose agents, TOOLUNIVERSE can be paired with specialized agents designed for specific scientific domains by integrating domain-specific tools. The following sections present three examples of building AI scientists: (i) LLM in-context use, (ii) MCP agent use, and (iii) specialized agents.

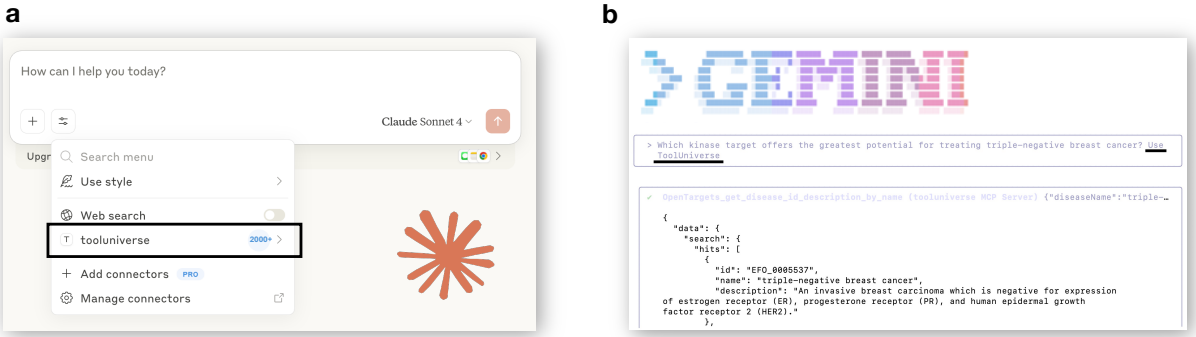



**Figure S1:** Building AI scientists on TOOLUNIVERSE. (a) Using TOOLUNIVERSE with a large language model in-context: the Claude desktop application registers TOOLUNIVERSE as a Model Context Protocol (MCP) connector, exposing its tools (here, more than 2,000) for selection and use directly within a chat session. (b) Using TOOLUNIVERSE with an AI agent: the Gemini CLI connects to TOOLUNIVERSE over MCP and autonomously calls tools to answer a research query (here, prioritizing kinase targets for triple-negative breast cancer), showing an issued tool call to a TOOLUNIVERSE MCP tool and its returned result.

## S4.1 Building an AI Scientist from an LLM or an LRM

We illustrate building an AI scientist from an LLM, such as Claude, together with TOOLUNIVERSE (Supplementary Figure S1a). The process involves three steps.

1. Install TOOLUNIVERSE with a single command and install the Claude desktop app.
2. Open the Claude Desktop and navigate to `Settings → Developer → Edit Config`. Set up the configurations as follows:

```
{
    "mcpServers": {
      "tooluniverse": {
```

```
            "command": "uv",
            "args": [
                "--directory",
                "path_to_ToolUniverse/src/tooluniverse",
                "run",
                "tooluniverse-mcp-claude"
            ]
        }
    }
}
```

3. Launch the Claude Desktop, select tools in TOOLUNIVERSE for the desired tasks.

## S4.2 Building an AI Scientist from an AI Agent

We demonstrate how to build an AI scientist using AI agents, such as the Gemini CLI, together with TOOLUNIVERSE (Supplementary Figure S1b). While AI agents can automatically leverage Tool Finder to identify the tools required for specific tasks, the process of creating an AI scientist involves two steps.

1. Install TOOLUNIVERSE with a single command and install the Gemini CLI.

2. Open the setting configuration file for Gemini CLI. Set up the configurations as follows:

```
{
  "mcpServers": {
    "tooluniverse": {
      "command": "uv",
      "args": [
        "--directory",
        "/path/to/your/gemini_running_env",
        "run",
        "tooluniverse-smcp-stdio"
      ]
    }
  },
}
```

## S4.3 Building an AI Scientist from a Specialized AI Agent

Specialized AI agents are trained on specific types of tasks, allowing them to become experts in particular domains, such as TxAgent [11] for precision therapeutics, GeneAgent [8] for gene-set analysis, and SpatialAgent [12] for spatial biology. TOOLUNIVERSE can be used not only by these specialized AI agents during inference but also as a real-world environment for agent training. For example, in TxAgent [11], TOOLUNIVERSE serves as the scientific environment in which TxAgent can interact. Through reinforcement learning, TxAgent learns how to use tools within TOOLUNIVERSE and manage complex therapeutic tasks.

## S5 Supplementary Note: Tools in TOOLUNIVERSE

TOOLUNIVERSE is an ecosystem that contains over 2,700 scientific tools covering essential scientific research domains. TOOLUNIVERSE provides built-in tool categories covering machine learning models, AI agents, software utilities, expert feedback systems, robotics, databases, embedding stores, data archives, and APIs, each serving specific computational and analytical requirements.

**Agentic tools.** Agentic tools operate autonomously to perform complex tasks using LLMs. Each agent is configurable with custom prompts and tool descriptions, supporting multiple backend models, including ChatGPT and Gemini. TOOLUNIVERSE includes agentic tools for literature summarization, code analysis, hypothesis generation, experiment planning, and results analysis. By defining the prompts and tool descriptions in the configuration file, one can quickly build an agentic tool.

**Scientific software package tools.** To support scientific coding, scientific package tools provide information about Python-based scientific computing libraries such as NumPy, Pandas, and SciPy. These tools offer installation instructions, usage examples, and documentation links, implementing dual-source data retrieval from PyPI APIs with local backup information.

**Database tools.** Database tools manage structured scientific resources such as DrugBank vocabulary datasets, clinical trial records, and molecular databases. They support integrations with tabular, hierarchical, XML-based, and graph-structured data. These tools provide capabilities for text-based search, field-level filtering, configurable result limits, and metadata return schemas. Built-in search, filtering, and indexing features can be reused when incorporating new databases.

**API integration tools.** API integration tools communicate with external scientific data sources using standard protocols such as RESTful APIs or GraphQL. Through these tools, users can access resources like FDA drug databases, OpenTargets disease–target associations, PubChem compound information, and many other databases, all with structured error handling and response validation. New tools can be incorporated by updating the API server URL, provided the common protocol is maintained.

**Expert feedback tools.** Expert feedback tools connect AI scientists with human experts, allowing AI scientists to request human suggestions or approval whenever necessary. This tool includes a server that connects the system with human experts, along with a user interface through which experts can provide responses. When a user calls the expert feedback tool, the request is redirected by the tool caller in TOOLUNIVERSE to a server, which forwards it to the human expert interface. Human experts can receive the request and provide their own insights and judgments. Their feedback is then sent back through the server as a tool response to the user. This approach supports consultation with human experts for complex scientific decisions and interpretations, combining automated analysis with expert validation.

**Machine learning tools.** Machine learning tools apply predictive and generative models to scientific use cases, such as disease–target scoring [13], disease-state prediction, gene-gene interaction [14, 15], gene dependency analysis [16], ADMET prediction [17], binding affinity prediction [18], and beyond. Since running environments for machine learning models often require specialized setups and hardware (e.g., GPUs), which can be difficult to deploy, TOOLUNIVERSE uses a remote registration scheme. This approach allows models to run on private servers while still being exposed as tools within TOOLUNIVERSE. New machine learning models can be quickly

integrated into TOOLUNIVERSE through remote tool registration.

**Embedding store tools.** Embedding store tools manage vectorized representations of scientific data. Scientific data is first transformed into embeddings using embedding models and then stored in a database. TOOLUNIVERSE employs FAISS to enable semantic search, similarity matching, and data retrieval over these embedding databases.

## S6 Supplementary Note: Pre-built Research Skills

To make TOOLUNIVERSE directly usable by experimental and computational biologists, the platform provides a library of pre-built research *skills*. Each skill is a natural-language-invokable workflow that orchestrates multiple tools to complete an end-to-end scientific task and return a reproducible result with provenance. As of this writing, TOOLUNIVERSE includes more than 130 research skills spanning genomics, transcriptomics and gene expression, proteomics, variant interpretation and clinical genetics, drug discovery and pharmacology, single-cell and spatial omics, microbiome and infectious disease, and structural biology. A user states a goal in natural language and the platform routes the request to the appropriate skill, which executes the underlying tools, performs the analysis, and returns results. Representative skills include RNA-seq differential expression with DESeq2 (`tooluniverse-rnaseq-deseq2`), drug-target validation (`tooluniverse-drug-target-validation`), comprehensive disease research (`tooluniverse-disease-research`), clinical variant interpretation with ACMG classification (`tooluniverse-variant-interpretation`), single-cell RNA-seq analysis (`tooluniverse-single-cell`), and GWAS-driven drug discovery (`tooluniverse-gwas-drug-discovery`). Skills follow a common specification and can be contributed and extended by the community. Supplementary Table S4 groups the available skills by application and data-modality area.

### S6.1 Worked examples

To make the skill workflow concrete, we trace two requests end to end—from the natural-language ask, through the tools invoked, to the returned result. All results below are from live tool executions.

**A single-call skill (sequence retrieval).** A user asks "get me the canonical TP53 protein sequence." The platform routes to `EnsemblSeq_get_id_sequence`, which resolves the Ensembl protein ID `ENSP00000269305` and returns the 393-amino-acid sequence (`MEEPQSDPSV...`), the canonical p53 isoform. This is the simplest skill shape: one natural-language request mapped to one validated tool call.

**An end-to-end analysis skill (RNA-seq differential expression).** A user asks "run differential expression on my airway RNA-seq data, dexamethasone-treated versus untreated, controlling for donor," providing a count matrix (public benchmark data: airway smooth-muscle cells from four donors, treated and untreated; [19], GEO GSE52778). The skill `tooluniverse-rnaseq-deseq2` loads the matrix (63,677 genes $\times$ 8 samples), builds the design `~ cell + dex`, and runs R DESeq2 through the `run_deseq2_analysis` tool (contrast treated vs untreated, $\alpha = 0.05$). It returns 4,028 differentially expressed genes at $\text{padj} < 0.05$ (2,211 up, 1,817 down); the most significant genes include the glucocorticoid-responsive *DUSP1* ($\log_2 FC = 2.95$), and *CRISPLD2*—the gene the original study followed up—is also strongly induced ($\log_2 FC = 2.63$, $\text{padj} = 4.4 \times 10^{-46}$). Chaining the induced gene set into the same

tool's GO-enrichment step (`enrichgo`) returns 58 enriched biological-process terms after redundancy reduction; alongside airway smooth-muscle terms, these include *cellular response to glucocorticoid stimulus* (BH-adjusted $p = 4.7 \times 10^{-3}$), recovering the expected mechanism of action. The entire analysis runs from a single natural-language request, with every number traceable to the tool that produced it.

| Step | Tool (operation) | Result |
|---|---|---|
| Sequence retrieval | `EnsemblSeq_get_id_sequence` | TP53 protein, 393 aa |
| Differential expression | `run_deseq2_analysis` (deseq2) | 4,028 DEGs (2,211 up, 1,817 down) |
| Top DEG | `run_deseq2_analysis` (deseq2) | *CRISPLD2*, $\log_2 FC = 2.63$, padj $= 4.4 \times 10^{-46}$ |
| Pathway enrichment | `run_deseq2_analysis` (enrichgo) | glucocorticoid response, adj. $p = 4.7 \times 10^{-3}$ |

## S7 Supplementary Note: Evaluation of Tools

To ensure the correctness and reliability of tools before their inclusion in TOOLUNIVERSE, we implemented a multi-step evaluation process. This evaluation was designed both to validate tool functionality and to ensure scientific utility. Each tool passes through the following step-by-step review before it is admitted to TOOLUNIVERSE:

1. **Automated validation.** For each tool, we generate diverse sample inputs covering typical use cases and edge conditions and record the outputs, and automated optimizer and checker tools test the tool against these samples, verifying that its behavior is consistent with its declared functionality and refining its description for accuracy and usability before expert review.

2. **Human-in-the-loop review.** Every tool undergoes human expert review before inclusion, and the form of this review depends on the tool's provenance. (i) For tools that wrap an authoritative data source (for example national or international resources such as the NIH and FDA), the source is itself authoritative, so the review verifies that the integration in TOOLUNIVERSE returns the same records as the original source rather than re-validating the underlying data, as for the openFDA tools. (ii) For tools backed by a peer-reviewed publication, the underlying method has already been validated in the literature, so the review confirms that the wrapped tool reproduces the published behavior on reference inputs, as for the Boltz-2 docking tool. (iii) For tools maintained by their original developers (for example a model served through the developer's own API), the tool is evaluated jointly with that team to confirm correct integration; this was the case for the ESM Cambrian (ESMC) protein language model, which was incorporated in collaboration with the EvolutionaryScale team. (iv) For all other tools, scientific experts perform a full review of representative input-output traces, assessing correctness, interpretability, and consistency with expected domain knowledge, as for TOOLUNIVERSE's in-house scientific-calculation tools (for example enzyme-kinetics and statistical-test utilities). In all cases, this human evaluation provides a safeguard against erroneous or misleading outputs.

3. **Decision and admission.** On the basis of the human review and the automated checks, the tool is either approved and registered into TOOLUNIVERSE, or returned for revision, to the

contributor or to the automated generation and optimization loop, and re-reviewed before a final decision.

4. **Regular maintenance and bug reporting.** For continued reliability of tools within TOOLUNIVERSE, we perform regular maintenance and monitoring. A structured bug reporting system allows issues to be identified by users and promptly addressed by the TOOLUNIVERSE team. These practices provide ongoing quality assurance and safeguard the long-term usability of the ecosystem.

Many tools in TOOLUNIVERSE are drawn from established, previously validated resources, and the provenance-dependent review above leverages that prior validation to reduce the risk of introducing spurious or unverified functionality into the ecosystem. These evaluation measures aim to ensure that tools incorporated into TOOLUNIVERSE meet standards of correctness, reproducibility, and scientific reliability, providing a foundation for AI-assisted discovery.

## S8 Supplementary Note: Quantitative Evaluation of TOOLUNIVERSE on End-to-End Tasks

This note describes the end-to-end benchmark used to evaluate TOOLUNIVERSE's higher-level modules (Tool Composer, Tool Discoverer, and Tool Optimizer). Because these modules act by assembling tools into complete workflows rather than as standalone predictors, we assess them through end-to-end task accuracy, the standard on which agentic systems such as Biomni [5] report.

**Benchmarks.** We evaluate on two LAB-Bench [20] subtasks: DbQA, which poses biomedical database questions that require querying authoritative resources, and SeqQA, which poses nucleotide- and protein-sequence questions with deterministic answers. To make the comparison to Biomni exact, we evaluate on the identical test items released by Biomni (60 DbQA and 70 SeqQA questions) and reuse Biomni's option construction and scoring procedure without modification, so every system is graded on the same questions under the same protocol; our reconstruction reproduces Biomni's released items and gold answers item for item.

**Systems and protocol.** We compare TOOLUNIVERSE-enabled agent systems with matched no-TOOLUNIVERSE agent systems and with Biomni. We run the controlled with-versus-without-TOOLUNIVERSE ablation in two independent agent settings, one built on Claude Code and one on Codex, so that the effect of adding TOOLUNIVERSE's tools is measured under two different base models; within each setting the two conditions share an identical agent and base model and differ only in whether the TOOLUNIVERSE tools are available. In the Claude setting, the TOOLUNIVERSE-enabled and no-TOOLUNIVERSE Claude Code agent systems both use Claude Opus 4.8. In the Codex setting (GPT-5.5), each question is sent to a fresh Codex process, with the TOOLUNIVERSE condition receiving the same tool-use instruction and the no-TOOLUNIVERSE condition receiving only the benchmark prompt; each Codex condition is run three independent times and reported as mean accuracy $\pm$ sample standard deviation. Biomni values are its reported accuracies on the same items. Answers are parsed and scored with Biomni's procedure (abstention or an unparseable response counts as incorrect).

**Results.** In a controlled ablation, equipping an agent with TOOLUNIVERSE's tools substantially improves its end-to-end task accuracy (Figure S2). Each TOOLUNIVERSE-enabled agent is compared against an identical agent and base model that differ only in whether TOOLUNIVERSE's tools are available, so the gain is attributable to the platform itself. It is large on DbQA, where accuracy rises from 56.7 to 78.3 for the Claude Code agent (a 21.6-point improvement) and from 81.1 to 92.8 for the Codex agent (an 11.7-point improvement); both agents also improve on SeqQA, from an already high baseline, rising from 94.3 to 99.5 for the Claude Code agent (a 5.2-point improvement) and from 91.4 to 96.2 for the Codex agent (a 4.8-point improvement). The effect reproduces across two agents built on different base models, Claude Opus 4.8 through Claude Code and Codex (GPT-5.5), indicating that the benefit is a general property of TOOLUNIVERSE and does not depend on a particular model or agent framework. Codex accuracies are means of three independent runs whose run-to-run standard deviations are small (at most 3.5 points, and 1.0 point once TOOLUNIVERSE is added on DbQA), confirming that the improvement is stable across runs.

The TOOLUNIVERSE-augmented agents also outperform Biomni, a purpose-built biomedical agent, on every subtask, evaluated on Biomni's own released items and scoring. Biomni reaches 74.4 on DbQA and 81.9 on SeqQA; the TOOLUNIVERSE-augmented Claude Code agent reaches

78.3 and 99.5 and the Codex agent 92.8 ± 1.0 and 96.2 ± 1.6, so a general-purpose agent equipped with TOOLUNIVERSE surpasses a specialized biomedical system on both tasks. The Claude numbers understate what the agent can do: its base model refuses some of the questions on safety grounds, and each refusal is counted as a wrong answer, which lowers the Claude scores whether or not TOOLUNIVERSE is available. Even under this penalty, the TOOLUNIVERSE-augmented Claude Code agent still outperforms Biomni on both subtasks.

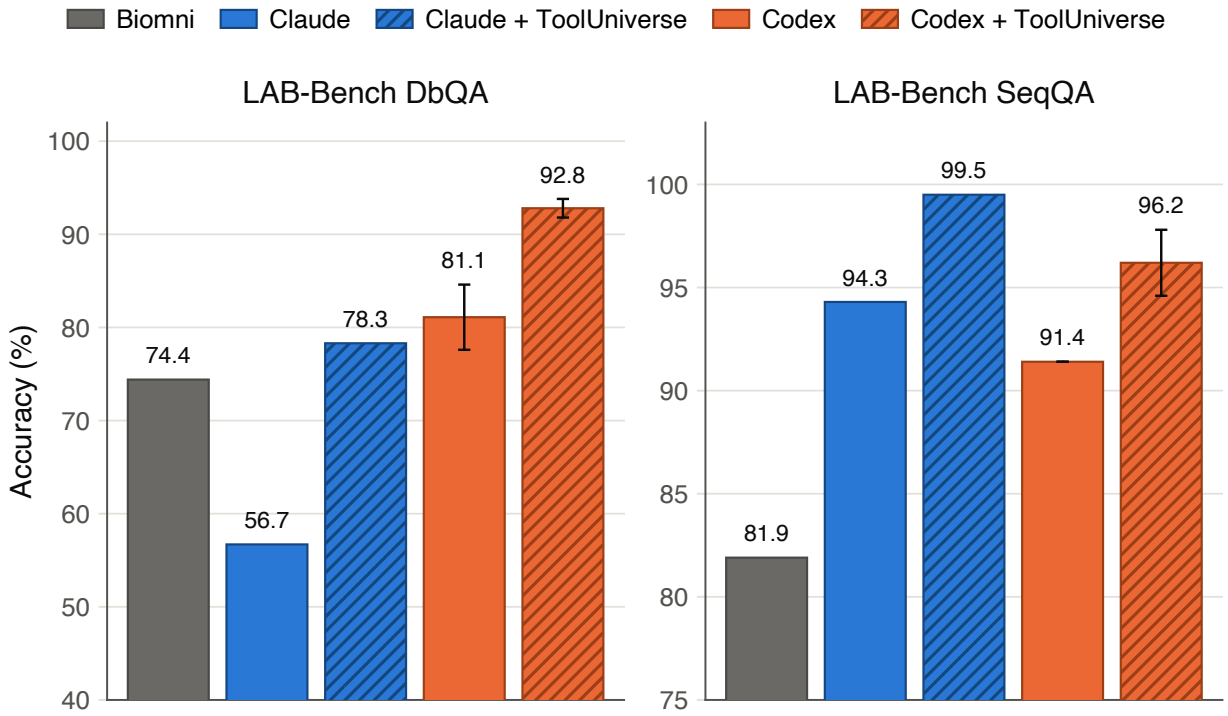



**Figure S2:** End-to-end accuracy (%) on LAB-Bench DbQA and SeqQA, shown as separate panels. Hatched bars are our TOOLUNIVERSE-augmented agents; solid bars are the baselines: the identical agent and base model run without TOOLUNIVERSE, and Biomni (gray). Within each agent, Claude Opus 4.8 (via Claude Code) and Codex (GPT-5.5), adding TOOLUNIVERSE (the hatched bar, right of its matched baseline) improves accuracy, and both TOOLUNIVERSE-augmented agents exceed Biomni on both subtasks; all systems are evaluated on the same publicly released items under the same scoring. Codex bars are the mean ± sample standard deviation over three independent runs (error bars); Claude and Biomni values are point estimates. A subset of questions are declined by the Claude base model (Claude Opus 4.8) under its safety policy and scored as incorrect, which lowers both Claude bars regardless of tool availability. Each panel's vertical axis begins above zero to resolve differences among the high-scoring systems.

## S9 Supplementary Note: Step-by-step real-world case studies

To illustrate how researchers use TOOLUNIVERSE in practice, we present three end-to-end case studies in which an AI scientist—an LLM connected to TOOLUNIVERSE—answers a real research question by autonomously selecting and chaining tools. Each case states the question in natural language, then follows the workflow as it composes tools across databases, predictive models, and analysis services, reporting the concrete result at each step. The workflows go beyond database lookup, integrating human genetics, functional-genomics screens (DepMap CRISPR essentiality [16]), protein structure (AlphaFold [21] and experimental PDB structures), cheminformatics and experimental bioactivity (ChEMBL [22]), and machine-learning property prediction (ADMET-AI [17]). The cases span (i) context-selective oncology-target assessment, (ii) structure- and prediction-guided chemical strategy, and (iii) clinical-trial biostatistics, and all results are from live tool executions. Across the three, the value of the workflow lies less in retrieving evidence than in reconciling evidence that points in different directions, and in refining an initial question as the evidence accrues. Strengths and limitations are summarized at the end.

### S9.1 Case 1: Is *BLM* a viable anticancer target, and is its chemical probe developable?

*Research question: "BLM loss-of-function predisposes to cancer by compromising genome maintenance, which raises a safety concern for inhibiting BLM. Is BLM nevertheless a context-selective anticancer target, and is its inhibitor ML216 a developable lead?"* The AI scientist approaches this in stages, refining the question as the evidence accrues.

1. **Resolve and annotate the gene.** `OpenTargets_multi_entity_search_by_query_string` (API Tool) maps *BLM* to `ENSG00000197299` ("BLM RecQ like helicase"), and `OpenTargets_get_target_gene_ontology_by_ensemblID` (API Tool) annotates a nuclear ATP-dependent 3′–5′ DNA helicase acting in DNA repair and homologous recombination (GO:0006281, GO:0043138, GO:0000724), with roles in replication-fork processing (GO:0031297) and telomere maintenance (GO:0061820, GO:0090656). A genome-maintenance helicase is the kind of gene whose loss promotes tumorigenesis, so its cancer-predisposition genetics counsel caution about inhibition rather than support it.

2. **Quantify the cancer-risk genetics.** `OpenTargets_get_diseases_phenotypes_by_target_ensembl` (API Tool) returns 464 disease associations. Bloom syndrome leads (Genomics England 0.96; ClinVar/EVA 0.95), followed by "inherited cancer-predisposing syndrome" (EVA 0.95) and a broad malignancy spectrum that includes colorectal cancer, acute myeloid and lymphoblastic leukemia, lymphoma, hereditary breast and ovarian cancer, and prostate carcinoma. These associations confirm that *BLM* loss is clinically consequential for genome instability.

3. **Classify the patient's variant.** `ClinVar_search_variants` (API Tool) returns 495 pathogenic *BLM* variants, among them `c.520C>T (p.Gln174Ter)`. For that variant (chr15:90,749,788, hg38), `GeneBe_classify_variant` (API Tool) returns an independent ACMG/AMP verdict of *Pathogenic* (score 12; criteria PVS1, PM2, PP5_Moderate; rs1895622788; transcript NM_000057.4). This is risk evidence, underscoring a genotoxicity concern for inhibition without validating *BLM* as a target.

4. **Test for a context-selective dependency.** `OpenTargets_get_target_depmap_essentiality` (API Tool) returns DepMap CRISPR gene-effect scores across 1,258 cancer cell lines. *BLM* is not pan-essential (flagged non-essential, with a mean gene effect of $-0.18$ and only 7.5% of lines, 94/1,258, dependent at $< -0.5$), yet its dependency concentrates in particular contexts: mature T- and NK-cell neoplasms ($-0.47$, $n$=8), cutaneous squamous-cell carcinoma ($-0.44$, $n$=5), and, at the tissue level, peripheral-nervous-system and neuroblastoma lines ($-0.33$, $n$=48). This is the turning point of the case: unlike the genetics, the screen reveals a quantitative, context-selective vulnerability that points to a possible therapeutic window in defined settings rather than to broad inhibition of a genome-stability factor.

5. **Retrieve the structure and check its reliability.** `alphafold_get_summary` (API Tool) returns the model for the 1,417-residue protein (`AF-P54132-F1`) together with its confidence: a mean pLDDT of 60.5, indicating confidently folded catalytic regions interspersed with low-confidence interdomain linkers. Querying `PDBe_get_uniprot_structure_coverage` (API Tool) then surfaces high-resolution experimental structures of the helicase core, including PDB `7AUC` (X-ray, 1.53 Å, residues ∼639–1290), `4CGZ` (a BLM–DNA complex, 3.2 Å), and `4O3M` (a ternary complex, 2.3 Å). Structure-based design should therefore be anchored on these experimental structures, with the AlphaFold model serving mainly to flag the unreliable linker regions.

6. **Ground the therapeutic context in the literature.** `EuropePMC_search_articles` (API Tool) shows that the clinically validated synthetic-lethal target in the RecQ family is not *BLM* but its paralog WRN, on which microsatellite-instable, mismatch-repair-deficient cancers depend, and that the WRN inhibitors HRO761 and VVD-133214 have entered clinical trials. The same search shows that ML216 itself has been used as a RecQ-helicase inhibitor with activity in MSI colorectal-cancer models. *BLM* functions in the alternative lengthening of telomeres (ALT), consistent with the neuroblastoma-enriched dependency seen above, neuroblastoma being a canonical ALT-positive lineage. These results frame *BLM* as a context-selective vulnerability hypothesis by analogy to the validated WRN/MSI precedent, with candidate contexts (mature T/NK neoplasms, cutaneous SCC, ALT-positive tumors) to test experimentally rather than a settled target.

7. **Find and triage the chemical matter.** `OpenTargets_get_chemical_probes_by_target_ensemblID` (API Tool) identifies ML216 but flags it as low quality (`isHighQuality` = false; ProbeMiner score 20; experimental origin), so the platform itself marks it a probe rather than a lead. `PubChem_get_CID_by_compound_name` and `PubChem_get_compound_properties_by_CID` (API Tools) resolve its structure (CID 49852229, a urea–thiadiazole of MW 383 Da), and `ADMETAI_predict_physicochemical_properties` and `ADMETAI_predict_toxicity` (ML Model Tools) find it drug-like (QED 0.66, no Lipinski violations) but flag a high predicted risk of drug-induced liver injury (DILI 0.99) alongside moderate hERG liability (0.56). Profiling five close analogs (`ChEMBL_search_similar_molecules`, 72–78% similarity), predicted hERG (0.45–0.67) and mutagenicity (AMES 0.12–0.31) vary across the series, whereas DILI remains near 0.99 (0.987–0.992) for every analog. This pattern is consistent

with a scaffold-associated DILI alert; it is a model-based hypothesis that would need experimental hepatotoxicity data to confirm, and if real would call for a scaffold change rather than peripheral substitution.

| ADMET-AI prediction (ML216) | Value | Interpretation |
|---|---|---|
| Molecular weight | 383 Da | within drug-like range |
| QED (drug-likeness) | 0.66 | favorable |
| Lipinski violations | 0 of 4 | passes rule of five |
| DILI (liver injury) | 0.99 | **high predicted risk—flag** |
| hERG (cardiac) | 0.56 | moderate predicted risk |
| AMES (mutagenicity) | 0.16 | low predicted risk |
| DILI across 5 close analogs | 0.987–0.992 | persistent; scaffold-associated (model hypothesis) |

**Outcome.** *BLM* is a genetics-validated cancer-predisposition gene whose loss supports a genome-maintenance role, so inhibition is not warranted by the genetics, which instead raise a genotoxicity concern. The evidence is better read as a context-selective vulnerability hypothesis: DepMap shows a modest dependency in defined settings (mature T/NK-cell neoplasms, cutaneous squamous-cell carcinoma, and neuroblastoma or ALT-positive lines), and the literature supplies a validated RecQ-family precedent in the paralog WRN for MSI/MMR-deficient cancers, together prioritizing specific contexts and biomarkers for experimental testing rather than establishing a target. ML216 is drug-like but platform-flagged as low quality, and its predicted liver-injury liability persists across close analogs, making it an early chemical probe rather than a developable lead. The value is a calibrated synthesis that keeps the safety, opportunity, and chemical-liability signals distinct and reconciles them into one testable hypothesis, each claim traceable to its tool.

## S9.2 Case 2: Assessing *OXTR* druggability and the chemical strategy for a CNS indication

*Research question: "Is OXTR (the oxytocin receptor) a druggable target, and does its pharmacology offer a credible chemical starting point for its emerging autism association?"* The AI scientist assembles a target dossier, finding that the straightforward "activate the receptor" reading needs qualification at several points.

1. **Resolve, annotate, and obtain structure.** `OpenTargets_multi_entity_search_by_query_string` maps *OXTR* to `ENSG00000180914`, and `OpenTargets_get_target_gene_ontology_by_ensemblID` identifies a plasma-membrane G-protein-coupled receptor that signals through a phospholipase-C/calcium pathway (GO:0004990, GO:0007200). `alphafold_get_summary` returns a confidently modeled transmembrane core (`AF-P30559-F1`, 389 residues, mean pLDDT 78.6), but `PDBe_get_uniprot_structure_coverage` (API Tools) surfaces experimental structures in both functional states: an agonist-bound active complex (PDB `7RYC`, cryo-EM 2.9 Å, *OXTR* bound to oxytocin with a heterotrimeric $G_q$ protein, and `7QVM` at 3.25 Å) and an inactive receptor (`6TPK`, X-ray 3.2 Å). Because GPCR design depends on ligand-bound, state-specific conformations, these experimental structures rather than the predicted model are the appropriate basis for design, and the active, agonist-bound state matches the pharmacology the indication requires.

2. **Map the disease landscape.** `OpenTargets_get_diseases_phenotypes_by_target_ensembl` (API Tool) returns 393 associations, spanning reproductive and uterine outcomes with approved-drug precedent as well as a distinct association with autism spectrum disorder. The latter is an emerging, hypothesis-generating direction rather than a clinically validated one.

3. **Confirm tractability and enumerate drugs.** `OpenTargets_get_target_tractability_by_ensemblID` classifies *OXTR* as a druggable-family GPCR with a ligand-bound structure and high-quality ligands, but its approved agents are peptides rather than small molecules, and non-peptide chemistry reaches only advanced-clinical stage. `OpenTargets_get_associated_drugs_by_target_ensemblID` (API Tools) returns nine agents, among them oxytocin, atosiban, and carbetocin (approved), retosiban and nolasiban (phase 3), and barusiban and epelsiban (phase 2). This establishes ligand precedent for reproductive and uterine indications, which does not de-risk an autism indication.

4. **Resolve mechanism.** `OpenTargets_get_drug_mechanisms_of_action_by_chemblId` (API Tool) shows the receptor is drugged in both directions: oxytocin is an agonist that induces labor, and atosiban an antagonist used as a tocolytic.

5. **Confirm chemical matter with experimental data.** Resolving the ChEMBL target (`ChEMBL_search_targets` $\rightarrow$ CHEMBL2049) and retrieving its activities (`ChEMBL_get_target_activities`, API Tools) returns measured binding affinities reaching single-digit nanomolar potency ($K_i$ as low as 3.2 nM), confirming that the receptor has well-characterized, high-affinity ligands.

6. **Reason about pharmacology and CNS druggability.** The pro-social hypothesis calls for receptor activation, but the clinical evidence is mixed. `EuropePMC_search_articles` shows that no oxytocin product is approved for the core social-communication symptoms of autism, that intranasal oxytocin remains investigational in recent reviews, and that trials are confounded by large placebo responses, including a randomized lead-in in which nearly half of participants met placebo-responder criteria. The medicinal-chemistry objective is therefore not simply "agonist good, antagonist bad" but a brain-penetrant agonist, partial agonist, or positive allosteric modulator with selectivity over the vasopressin receptors. To test whether a brain-penetrant non-peptide chemotype is even attainable, the workflow profiles nolasiban (`PubChem_get_CID_by_compound_name`, CID 52947354) with `ADMETAI_predict_physicochemical_properties` and `ADMETAI_predict_BBB_penetrance` (ML Model Tools), finding it drug-like (QED 0.87) with high predicted blood-brain-barrier penetrance (0.93). That figure indicates plausibility, not proven central exposure, and in any case nolasiban is an antagonist: brain-penetrant non-peptide *OXTR* chemistry is feasible, but the available CNS-capable chemotype blocks the receptor rather than activating it.

**Outcome.** Rather than a naive "repurpose an approved drug" suggestion, TOOLUNIVERSE delivers a mechanistically careful conclusion. *OXTR* is a tractable, well-liganded GPCR with experimental structures in both active and inactive states, clinically validated for reproductive and uterine

indications, whereas its autism association remains an emerging hypothesis with inconsistent clinical support. Central engagement is chemically plausible, but the pro-social hypothesis requires agonism while the only brain-penetrant chemotype on hand is an antagonist. Integrating mechanism, experimental affinity and structure, and machine-learning prediction, the platform defines a precise objective—a brain-penetrant *OXTR* agonist or positive allosteric modulator, selective over the vasopressin receptors and with confirmed central exposure—and so avoids the pharmacologically wrong repurposing call a drug-name lookup would invite.

## S9.3 Case 3: Clinical-trial safety biostatistics for BCG vaccination and adverse-event severity

*Research question: "In the public BCG-CORONA healthcare-worker trial, is BCG vaccination associated with higher adverse-event severity after adjusting for patient-interaction frequency?"* Unlike Cases 1 and 2, this question calls for statistical computation on raw tabular data. The AI scientist uses `clinical_trial_ae_severity_test` (Software Tool), which merges the trial's demographics (DM) and adverse-events (AE) tables and models severity by treatment arm. Following the tool's convention, each subject's outcome is the maximum severity grade (AESEV) across all of their adverse events, and the cohort is those subjects with at least one adverse-event record, obtained by an inner join of AE onto DM. Data are from the publicly available BCG-CORONA trial (https://zenodo.org/records/12737228).

1. **Construct the analysis cohort** (`prepare` mode): the inner join yields 791 subjects with evaluable AE records, each reduced to a single maximum-severity value (distribution {grade 1: 328, 2: 402, 3: 43, 4: 18}).

2. **Test for an unadjusted association** (`chi-square` mode): the treatment-by-severity table shows a significant association ($\chi^2 = 10.12$, dof $= 3$, $p = 0.018$).

3. **Fit an adjusted model** (`ordinal` mode) with patient-interaction covariates: a multivariable ordinal logistic (proportional-odds) regression gives an odds ratio of 1.53 (95% CI 1.16–2.01, $p = 0.0024$) for higher severity with BCG vaccination, and the covariates are not significant. Because this is a proportional-odds model, the single odds ratio assumes a common effect across severity thresholds, an assumption that should be checked before the estimate is over-interpreted.

| Step / test | Tool mode | Result |
|---|---|---|
| Cohort construction | `prepare` | 791 AE-evaluable subjects; AESEV {1:328, 2:402, 3:43, 4:18} |
| Unadjusted association | `chi-square` | $\chi^2 = 10.12$, dof $= 3$, $p = 0.018$ |
| Adjusted model | `ordinal` | OR $= 1.53$ (95% CI 1.16–2.01), $p = 0.0024$ |

**Outcome.** The unadjusted test and the adjusted model concur that, in this cohort, BCG assignment is associated with higher maximum adverse-event severity (adjusted OR 1.53); the two are not independent, since they share the outcome and overlapping data. Because the cohort is restricted to participants with adverse-event records and the endpoint is a derived per-subject maximum, the finding is best read as a reproducible safety signal from a secondary analysis rather than a definitive effect. As a demonstration, TOOLUNIVERSE performs the analysis—cohort construction, endpoint

derivation, and modeling—directly on raw trial tables, making the cohort and endpoint definitions explicit.

## S9.4 Strengths and limitations illustrated by the case studies

*Strengths.*

- **From a natural-language question, end to end.** Each case begins with a plain-language question that the AI scientist answers by autonomously selecting and chaining tools, reporting a concrete result at each step without user-written code.
- **Composition across modalities.** One workflow spans human genetics, functional-genomics screens (DepMap), protein structure (AlphaFold and experimental PDB structures), cheminformatics and experimental bioactivity (ChEMBL), machine-learning prediction (ADMET-AI), and statistics, assembling in a single run the evidence that would otherwise demand many separate resources and custom code.
- **Reconciling conflicting evidence.** The value is adjudication, not retrieval: in the *BLM* case the workflow holds a safety signal, an opportunity signal, and a predicted chemical liability apart, then reconciles them into a testable, biomarker-defined hypothesis rather than treating a predisposition gene as an automatic target.
- **Calibrated conclusions.** It avoids overclaiming, returning *BLM* as a hypothesis rather than a target, superseding a low-confidence predicted structure with experimental ones, treating a predicted BBB score as plausibility rather than exposure, and reporting a persistent predicted-DILI pattern as a model hypothesis to confirm.
- **Reproducibility and provenance.** Every step is a typed call through one interface, so each result carries the name and arguments of the tool that produced it; with versioned releases and optional audit logging, a workflow can be inspected and re-run.
- **Cross-checking that catches errors.** Combining evidence types exposes what a single lookup misses: in the *OXTR* case, a brain-penetrant hit (nolasiban) looks like a repurposing candidate until its mechanism reveals an antagonist where the hypothesis needs activation, a mismatch the workflow flags before it becomes a recommendation.

*Limitations.* TOOLUNIVERSE orchestrates rather than certifies: it composes existing tools rather than re-deriving them, so any conclusion is bounded by the underlying databases and models. Machine-learning outputs such as an ADMET-AI score, and association evidence, are estimates to weigh by provenance and validation domain and confirm experimentally, not measurements; a dependency score prioritizes tumor contexts but does not establish pharmacologic tractability, tumor-selective toxicity, or an adequate therapeutic index; and quantitative results still need domain judgment, including whether a derived endpoint captures the intended construct (a per-subject maximum over all adverse events), which cohort an inner join defines, and an odds ratio's direction relative to its reference level (reported as $0.65$ or its reciprocal $1.53$). TOOLUNIVERSE mitigates these with a human-expert curation gate, recorded provenance, and human-in-the-loop checkpoints, and it makes the evidence, computations, and predictions explicit so these judgments are transparent. It does not by itself certify scientific correctness, and the scientist remains in the loop.

Tool Description Schema in TOOLUNIVERSE

**Name**: The unique identifier for the tool.

**Description**: A clear and concise summary of the tool's purpose and functionality.

**Parameters**: A list of arguments that the tool accepts. Each argument has the following properties:

- **Argument Name**: The name of the parameter.
- **Argument Type**: The expected data type for the parameter's value (e.g., string, integer, boolean).
- **Argument Description**: A detailed explanation of what the parameter represents and its purpose.
- **Required**: A boolean value indicating whether the parameter is mandatory for the tool to execute.

**Return Schema**: A description of the structure and data types of the output returned by the tool upon successful execution.

**Figure S3:** The tool description schema in TOOLUNIVERSE is consistent across all tools, regardless of their diverse backends.

```
[
    {
        "type": "ChEMBLTool",
        "name": "ChEMBL_search_similar_molecules",
        "description": "Search for molecules similar to a given SMILES, chembl_id, or compound or drug name, using the ChEMBL Web Services.",
        "parameter": {
            "type": "object",
            "properties": {
                "query": { "type": "string", "description": "SMILES string, chembl_id, or compound or drug name." },
                "similarity_threshold": { "type": "integer", "description": "Similarity threshold (0–100).", "default": 80 },
                "max_results": { "type": "integer", "description": "Maximum number of results to return.", "default": 20 }
            },
            "required": ["query"]
        }
    }
]
```


**Figure S4:** One example of a tool description in TOOLUNIVERSE.

Interaction schema in TOOLUNIVERSE

**Name**: The name of the tool or operation to be called.

**Parameters**: A list of arguments that the tool accepts. Each argument has the following properties:

- **Argument Name**: The name of the parameter.
- **Argument Value**: The value for the parameter provided by the client.

**Figure S5:** The universal interaction schema for all tools and operations within TOOLUNIVERSE.

```python
def run(self, arguments):
    query = arguments.get("query")
    similarity_threshold = arguments.get("similarity_threshold", 80)
    max_results = arguments.get("max_results", 20)

    if not query:
        return {"error": "`query` parameter is required."}
    return self._search_similar_molecules(query, similarity_threshold, max_results)
```

**Figure S6:** Code example of a tool operation implementation used by the Tool Caller during execution.

```python
from tooluniverse.tool_registry import register_tool

@register_tool('MyTool', config={
    "name": "my_tool",
    "type": "MyTool",
    "description": "Does something useful",
    "parameter": {
        "type": "object",
        "properties": {
            "input": {"type": "string", "description": "Your input"}
        },
        "required": ["input"]
    }
})
class MyTool:
    def __init__(self, tool_config=None):
        self.tool_config = tool_config

    def run(self, arguments):
        return {"result": f"Processed: {arguments['input']}", "success": True}
```

**Figure S7:** Example code demonstrating how to register a local tool with Tool Manager and add it to TOOLUNIVERSE.

```python
from tooluniverse.mcp_tool_registry import register_mcp_tool, start_mcp_server

@register_mcp_tool(
    tool_type_name="my_analyzer",
    config={
        "description": "Analyzes data and returns results",
        "parameter_schema": {
            "type": "object",
            "properties": {
                "data": {"type": "string", "description": "Data to analyze"}
            },
            "required": ["data"]
        }
    },
    mcp_config={
        "server_name": "My Analysis Server",
        "port": 8001,
        "host": "0.0.0.0"
    }
)
class MyAnalyzer:
    def run(self, arguments):
        data = arguments.get('data', '')
        return {"result": f"Analyzed: {data}", "success": True}
```

**Figure S8:** Example code demonstrating how to register a remote tool with Tool Manager and add it to TOOLUNIVERSE.

```python
def compose(arguments, tooluniverse, call_tool):
    """Search literature and generate summary"""
    topic = arguments['research_topic']

    literature = {}
    literature['pmc'] = call_tool('EuropePMC_search_articles', {'query': topic, 'limit': 5})
    literature['openalex'] = call_tool('openalex_literature_search', {'search_keywords': topic, 'max_results': 5})
    literature['pubtator'] = call_tool('PubTator3_LiteratureSearch', {'text': topic, 'page_size': 5})

    summary = call_tool('MedicalLiteratureReviewer', {
        'research_topic': topic, 'literature_content': str(literature),
        'focus_area': 'key findings', 'study_types': 'all studies',
        'quality_level': 'all evidence', 'review_scope': 'rapid review'
    })

    return summary
```

**Figure S9:** This example demonstrates a composable tool, built with the Tool Composer, that runs multiple literature search tools concurrently, followed by a summary agent that synthesizes the results. The Tool Composer enables the combination of multiple tools from TOOLUNIVERSE in diverse ways, such as in parallel, sequentially, or in loops, enabling multi-tool collaboration.

| Tool category | Description |
|---|---|
| ML models | Tools that apply machine learning algorithms to tasks like prediction, classification, or generation. Examples include foundation models, language models, and reasoning models. |
| Agents | Tools that operate autonomously to perceive environments, make decisions, and take actions toward goals. Examples include research planning agents, hypothesis generation agents, and data analysis agents. |
| Scientific packages | Software packages engineered to facilitate diverse scientific tasks, experiments, and data analysis workflows. Examples include computation packages, biomedical libraries, and scientific simulators. |
| Automation | Tools involving physical or simulated machines capable of sensing, reasoning, and acting in the world. Examples include lab automation tools and instrument control policies. |
| Workflows | Tools that enable complex, multi-step scientific workflows. Examples include orchestration schedulers and RAG pipelines. |
| Visualization | Tools that facilitate the display and communication of scientific data and results. Examples include dashboards and plot and chart generators. |
| Feedback and safety | Tools that incorporate evaluations or input from human experts to ensure safety and goal alignment. Examples include privacy guardrails, security checklists, and human-in-the-loop tools. |
| Local and remote datasets | Tools that store, manage, and query structured or semi-structured data, including relational, tabular, and hierarchical datasets. Examples include knowledge bases and typed API clients. |
| Experimentation | Tools that support the design and management of experiments. Examples include protocol generators and LIMS integration tools. |
| Search and embeddings | Tools that store and retrieve vectorized representations of data for use in machine learning tasks. Examples include vector search tools and embedding generators. |

**Table S1:** Types of tools in TOOLUNIVERSE that TOOLUNIVERSE-powered AI scientists can use.

| Tool category | Number | Examples of tools in TOOLUNIVERSE |
|---|---|---|
| ML models | 89 | `ADMETAI_predict_toxicity`, `SpliceAI_predict_splice`, `IEDB_predict_mhci_binding` |
| Agents | 45 | `HypothesisGenerator`, `CodeQualityAnalyzer`, `MedicalLiteratureReviewer` |
| Scientific packages | 179 | `get_biopython_info`, `get_rdkit_info`, `get_scanpy_info` |
| Automation | 4 | `mcp_auto_loader_boltz`, `mcp_auto_loader_txagent`, `mcp_auto_loader_esm` |
| Workflows | 16 | `BiomarkerDiscoveryWorkflow` `ComprehensiveDrugDiscoveryPipeline` `ToolDiscover` |
| Visualization | 3 | `visualize_protein_structure_3d` `visualize_molecule_2d` `visualize_molecule_3d` |
| Feedback and safety | 7 | `consult_human_expert`, `get_expert_response`, `get_expert_status` |
| Local and remote datasets | 2,421 | `ChEMBL_get_molecule`, `HPA_get_comprehensive_gene_details_by_ensembl_id`, `FDA_get_active_ingredient_info_by_drug_name`, `OpenTargets_get_associated_targets_by_disease_efoId` |
| Search and embeddings | 9 | `Tool_Finder`, `Tool_RAG`, `embedding_database_search` |
| Experimentation | 4 | `ExperimentalDesignScorer`, `ProtocolOptimizer`, `extract_clinical_trial_outcomes` |
| **Unique tools** | **2,777** | *Total number of unique tools across all categories* |

**Table S2:** Number of tools and example tools for each category in TOOLUNIVERSE, from a direct enumeration of the open-source repository (`main` branch) on 7 July 2026. Each tool is assigned to a single primary category from its implementation type, name, and source, so the category counts sum to the total of 2,777 unique tools. TOOLUNIVERSE is continually expanded with new tools.

To characterize coverage of the tool library by data modality and application area, Supplementary Table S3 reports the number of tools in each area. Because a tool may serve several modalities, it is counted in every area to which it applies.

| Data-modality / application area | Tools | Representative resources |
|---|---|---|
| Drugs & pharmacology | 474 | FDA drug labels, PharmGKB, DailyMed |

| | | |
|---|---|---|
| Clinical, disease & phenotypes | 289 | ClinicalTrials.gov, Open Targets, HPO / Orphanet |
| Genomes, sequences & browsers | 222 | Ensembl, UCSC Genome Browser, ENA |
| Proteins, sequence, function & proteomics | 193 | UniProt, InterPro, PRIDE |
| Tooling, methods & infrastructure | 193 | Agentic tools, Tool Finder, software packages |
| Macromolecular structure & prediction | 189 | RCSB PDB, AlphaFold, PDBe |
| Genetic variation, GWAS & population genetics | 184 | gnomAD, GWAS Catalog, ClinVar |
| Pathways & molecular networks | 169 | Reactome, KEGG, STRING |
| Chemistry & small molecules | 156 | ChEMBL, PubChem, ADMET-AI |
| Literature & scholarly data | 143 | PubMed, Europe PMC, OpenAlex |
| Transcriptomics & gene expression | 137 | GTEx, GEO, Human Protein Atlas |
| Earth, space, ecology & socioeconomic | 120 | NASA, Open-Meteo, GBIF |
| Annotation, ontologies & enrichment | 111 | Gene Ontology, Enrichr, OLS |
| Cancer genomics & oncology | 106 | cBioPortal, GDC, OncoKB |
| Model organisms & comparative genomics | 87 | SGD, ZFIN, Alliance of Genome Resources |
| Metabolomics & lipidomics | 78 | MetaboLights, GNPS, LIPID MAPS |
| Epigenomics & gene regulation | 78 | ENCODE, JASPAR, ChIP-Atlas |
| Imaging & neuroscience | 77 | CryoET Portal, OpenNeuro, Allen Brain |
| Microbiome & infectious disease | 54 | MGnify, GTDB, BV-BRC |
| Immunology | 43 | IEDB, VDJdb, IMGT |

**Table S3:** Coverage of the TOOLUNIVERSE tool library by data modality and application area (open-source repository `main` branch, 7 July 2026). Categories are not mutually exclusive: a tool serving several modalities is counted in each, so the column sums to more than the number of unique tools (2,777 unique; 3,103 category memberships). Assignments were generated programmatically from each tool's resource and description. Supplementary Table S4 lists 130 research skills, grouped by application areas and data modalities.

| **Skill** | **Description** |
|---|---|
| **Genomics, sequences and variants** | |

| Skill | Description |
|---|---|
| tooluniverse-acmg-variant-classification | Systematic ACMG/AMP germline variant classification with all 28 criteria (PVS1, PS1-4, PM1-6, PP1-5, BA1, BS1-4, BP1-7) for... |
| tooluniverse-variant-analysis | VCF and variant analysis: parsing, annotation, classification (synonymous, missense, frameshift, stop_gained), VAF filtering,... |
| tooluniverse-variant-interpretation | Clinical variant interpretation from raw variant calls to ACMG-classified recommendations with structural impact analysis |
| tooluniverse-variant-functional-annotation | Functional annotation of protein variants: ProtVar structural/functional context, ClinVar clinical classifications, gnomAD... |
| tooluniverse-variant-to-mechanism | End-to-end variant-to-mechanism analysis: trace a variant (rsID/coordinates) through regulatory context, target gene(s), molecular... |
| tooluniverse-structural-variant-analysis | Structural variant (SV) clinical interpretation: deletions, duplications, inversions, translocations, complex rearrangements |
| tooluniverse-regulatory-variant-analysis | Non-coding/regulatory variant interpretation: GWAS association lookup, eQTL evidence (GTEx), chromatin state (ENCODE), regulatory... |
| tooluniverse-cancer-variant-interpretation | Clinical interpretation of somatic cancer mutations for precision oncology |
| tooluniverse-sequence-analysis | Biological sequence analysis: gene/protein sequence retrieval (NCBI, Ensembl, UniProt), nucleotide/protein search, ortholog... |
| tooluniverse-sequence-retrieval | Retrieve DNA/RNA/protein sequences from NCBI and ENA with disambiguation |
| tooluniverse-fastq-qc | FASTQ quality control and adapter/quality-trimming decisions with local NGS tools: run FastQC on raw reads, summarize a project... |
| tooluniverse-primer-design | PCR / qPCR primer and oligo design: design forward/reverse primers for a target region (SantaLucia nearest-neighbor... |
| tooluniverse-molecular-cloning | Molecular cloning assembly design: Gibson Assembly (overlap design for seamless multi-fragment joining) and Golden Gate Assembly... |
| tooluniverse-comparative-genomics | Cross-species gene comparison and ortholog analysis |
| tooluniverse-model-organism-genetics | Cross-species genetic analysis using model organism databases (MGI mouse, ZFIN zebrafish, FlyBase fruit fly, WormBase worm, SGD... |
| tooluniverse-plant-genomics | Plant genomics and biology research: PlantReactome pathways, Ensembl Plants gene structure, POWO species taxonomy, UniProt... |
| tooluniverse-rare-disease-genomics | Rare disease genomics: disease identification (Orphanet), causative gene discovery, gene-disease validity (GenCC), variant... |

| Skill | Description |
|---|---|
| tooluniverse-cancer-genomics-tcga | TCGA/GDC cancer genomics analysis: cohort construction, clinical metadata retrieval, somatic mutation frequencies, survival... |
| tooluniverse-microbial-genome-characterization | Genome-ASSEMBLY discovery, QC, and replicon mapping for any organism (bacteria, archaea, fungi, and beyond) using NCBI Datasets |
| **GWAS, population and statistical genetics** | |
| tooluniverse-gwas-drug-discovery | Transform GWAS signals into drug targets and repurposing opportunities |
| tooluniverse-gwas-finemapping | Statistical fine-mapping of GWAS loci using credible sets (SuSiE, FINEMAP) and locus-to-gene scoring (Open Targets L2G) |
| tooluniverse-gwas-snp-interpretation | Interpret a single GWAS SNP across multiple databases: GWAS Catalog hits, LD/haplotype context, eQTL evidence, regulatory... |
| tooluniverse-gwas-study-explorer | Compare GWAS studies, perform meta-analyses across cohorts, and assess signal replication |
| tooluniverse-gwas-trait-to-gene | Discover causal genes for diseases/traits from GWAS data using Open Targets L2G (locus-to-gene) scoring: integrates eQTL,... |
| tooluniverse-phewas | Cross-ancestry / cross-biobank phenome-wide association (PheWAS) and replication |
| tooluniverse-polygenic-risk-score | Build and interpret polygenic risk scores (PRS) for complex diseases using GWAS summary statistics |
| tooluniverse-population-genetics | Population genetics analysis: allele frequencies (gnomAD, 1000 Genomes), Hardy-Weinberg equilibrium testing, Fst between... |
| tooluniverse-population-genetics-1000genomes | Population genetics using the 1000 Genomes Project (IGSR): superpopulation/population search, sample metadata, variant frequencies... |
| tooluniverse-mendelian-randomization | Mendelian randomization (MR) causal inference: does an exposure, risk factor, or biomarker CAUSALLY affect a disease/outcome,... |
| tooluniverse-gene-disease-association | Gene-disease association analysis across DisGeNET, OpenTargets, Monarch, OMIM, GenCC, Orphanet |
| tooluniverse-pathway-disease-genetics | Connect GWAS variants to biological pathways and druggable targets |
| **Transcriptomics and functional genomics** | |
| tooluniverse-rnaseq-deseq2 | RNA-seq differential expression analysis with DESeq2, edgeR, and limma-voom: DEG lists, fold changes, dispersion estimation,... |
| tooluniverse-single-cell | Single-cell RNA-seq analysis with scanpy/anndata: h5ad data loading, scRNA-seq quality control and QC gating (n_genes_by_counts,... |
| tooluniverse-spatial-transcriptomics | Spatial transcriptomics analysis: Visium, MERFISH, seqFISH, Slide-seq |

| Skill | Description |
|---|---|
| tooluniverse-expression-data-retrieval | Retrieve gene expression and omics datasets from ArrayExpress and BioStudies with gene disambiguation and quality assessment |
| tooluniverse-gene-enrichment | Gene-set enrichment analysis: GO (Biological Process, Molecular Function, Cellular Component), KEGG, Reactome pathway enrichment... |
| tooluniverse-functional-genomics-screens | Interpret hits from CRISPR-KO/CRISPRi/shRNA screens by integrating DepMap essentiality, gnomAD constraint scores, pathway context... |
| tooluniverse-crispr-screen-analysis | Analyze CRISPR-Cas9 genetic screens: MAGeCK gene-level scores, sgRNA count QC, replicate correlation, hit prioritization, and... |
| tooluniverse-gene-regulatory-networks | Gene regulatory network analysis: TF-target inference (JASPAR motifs, ChIP-seq), motif scanning, eQTL integration, perturbation... |
| tooluniverse-noncoding-rna | Non-coding RNA analysis: miRNAs (miRBase, miRDB targets), lncRNAs (LNCipedia, RNAcentral), circRNAs, snoRNAs, and other ncRNA classes |
| tooluniverse-epigenomics | Genomics and epigenomics analysis: DNA methylation (CpG, 5mC, 5hmC, bisulfite, RRBS), m6A RNA modification (MeRIP-seq), ChIP-seq... |
| tooluniverse-epigenomics-chromatin | Histone-modification ChIP-seq, ATAC-seq accessibility, chromatin state, and TF binding analysis from ENCODE, Roadmap Epigenomics,... |
| tooluniverse-regulatory-genomics | Transcription factor binding, cis-regulatory elements (cCREs), chromatin accessibility, and regulatory annotation using JASPAR... |
| **Proteomics, protein structure and design** | |
| tooluniverse-proteomics-analysis | Mass-spec proteomics analysis: protein identification, quantification (LFQ, TMT, iTRAQ), differential expression (tumor vs normal,... |
| tooluniverse-proteomics-data-retrieval | Find and retrieve proteomics datasets from MassIVE and ProteomeXchange |
| tooluniverse-protein-interactions | Protein-protein interaction (PPI) network analysis: STRING (predicted + experimental), BioGRID (curated), SASBDB (small-angle... |
| tooluniverse-protein-structure-prediction | Protein 3D structure prediction from sequence: ESMFold de novo prediction, AlphaFold database retrieval, experimental structures... |
| tooluniverse-protein-structure-retrieval | Protein structure retrieval from RCSB PDB, PDBe, and AlphaFold with disambiguation, quality assessment (resolution, R-factor,... |
| tooluniverse-protein-therapeutic-design | AI-guided de novo protein design: RFdiffusion backbone generation, ProteinMPNN sequence design, structure validation (pLDDT, pTM,... |

| Skill | Description |
|---|---|
| tooluniverse-protein-modification-analysis | Post-translational modification (PTM) analysis: phosphorylation, ubiquitination, acetylation, glycosylation, methylation |
| tooluniverse-protein-lof-mechanism | Propose the mechanism by which a missense variant causes loss-of-function (LoF), synthesizing evidence from 5 independent layers:... |
| tooluniverse-protein-sae-variant-interpretation | Interpret a missense variant via ESMC-6B Sparse Autoencoder (SAE) feature activations |
| tooluniverse-protein-structural-annotation-pdb | Given a PDB structure, produce a per-residue annotation table: which residues sit at a binding interface (vs a partner chain),... |
| tooluniverse-residue-functional-mechanism-interpretation | Given a set of residues in a protein, explain WHY they are functionally critical by combining structural context (binding... |
| tooluniverse-structural-proteomics | Structural biology plus proteomics integration for drug target validation |
| tooluniverse-variant-predictor-dms-validation | Validate a variant-effect predictor (AlphaMissense, ESM-C SAE, ESM logits, EVE, conservation scores, or any per-variant numeric... |
| tooluniverse-peptide-target-deorphanization | Find the real protein target(s) of a peptide from its sequence: peptide target deorphanization / off-target identification, for... |
| tooluniverse-electron-microscopy | Search and analyze electron microscopy data: cryo-EM density maps (EMDB), fitted atomic models (PDB), raw micrograph datasets... |
| tooluniverse-computational-biophysics | Solve quantitative problems in biophysics: pharmacokinetics (PK volume of distribution, clearance, half-life), epidemiology (R0,... |
| **Metabolomics, lipidomics and enzymology** | |
| tooluniverse-metabolomics | Metabolomics research: metabolite identification, study analysis, and database searches across HMDB, MetaboLights, Metabolomics... |
| tooluniverse-metabolomics-analysis | Analyze metabolomics data end-to-end: metabolite identification, quantification (TIC normalization, batch correction),... |
| tooluniverse-metabolomics-pathway | Metabolomics pathway analysis: metabolite identification (HMDB, KEGG, ChEBI), pathway mapping (Reactome, KEGG, MetaCyc), disease... |
| tooluniverse-lipidomics | Lipid analysis and lipid-disease associations using LIPID MAPS classification, HMDB metabolite data, KEGG/Reactome lipid pathways... |
| tooluniverse-natural-product-dereplication | Dereplicate a putative natural product and assign its chemical taxonomy |
| tooluniverse-enzyme-kinetics | Enzyme kinetics: Michaelis-Menten Km, Vmax, kcat (turnover), and kcat/Km (catalytic efficiency / specificity constant) from... |

| Skill | Description |
|---|---|
| **Chemistry and small-molecule discovery** | |
| tooluniverse-admet-prediction | Comprehensive ADMET (Absorption, Distribution, Metabolism, Excretion, Toxicity) profiling for drug candidates |
| tooluniverse-binder-discovery | Discover novel small-molecule binders for protein targets using structure-based and ligand-based screening |
| tooluniverse-chemical-compound-retrieval | Retrieve chemical compound data from PubChem and ChEMBL with disambiguation, cross-referencing, and stereochemistry handling |
| tooluniverse-chemical-safety | Chemical safety and toxicology assessment integrating ADMET-AI predictions, CTD toxicogenomics, PubChemTox experimental data,... |
| tooluniverse-chemical-sourcing | Find commercial sources for chemical compounds: PubChem/ChEMBL identity resolution then vendor catalog search across ZINC,... |
| tooluniverse-small-molecule-discovery | Small molecule identification, characterization, and procurement: PubChem, ChEMBL, BindingDB, ADMET-AI, SwissADME, eMolecules, Enamine |
| tooluniverse-organic-chemistry | Organic chemistry reasoning guide for reaction product prediction, mechanism analysis (electrophilic/nucleophilic substitution,... |
| tooluniverse-inorganic-physical-chemistry | Inorganic chemistry, physical chemistry, and materials science: crystal structures, coordination chemistry, lattice parameters,... |
| tooluniverse-gpcr-structural-pharmacology | GPCR receptor pharmacology: agonist/antagonist/inverse-agonist/biased-agonist classification, GPCRdb structural data,... |
| tooluniverse-network-pharmacology | Compound-target-disease network construction and analysis for drug repurposing, polypharmacology discovery, and multi-target drug... |
| tooluniverse-drug-synergy | Drug-combination synergy analysis: quantify whether two drugs together are synergistic, additive, or antagonistic using the... |
| tooluniverse-dose-response | Dose-response / concentration-response curve fitting: IC50, EC50, Hill slope, Emax/Emin efficacy, and relative potency from paired... |
| **Drug development and pharmacology** | |
| tooluniverse-drug-research | Comprehensive drug profiling: mechanism, primary/secondary targets, drug interactions, clinical-trial status, adverse events... |
| tooluniverse-drug-mechanism-research | Trace drug mechanism of action: primary target to downstream signaling to pathway perturbation to tissue/organ effect to clinical outcome |
| tooluniverse-drug-target-validation | Quantitative drug-target validation pipeline |
| tooluniverse-drug-repurposing | Identify drug repurposing candidates via target-based, compound-based, and disease-based strategies |
| tooluniverse-drug-drug-interaction | Assess drug-drug interactions: CYP metabolic interactions (substrate/inhibitor/inducer), transporter (P-gp, BCRP, OATP) effects,... |

| Skill | Description |
|---|---|
| tooluniverse-drug-regulatory | Drug regulatory and approval research: FDA substance registry, ATC/EPC classification, EMA decisions, generic-drug status, FDA... |
| tooluniverse-target-research | Comprehensive drug-target intelligence: tissue expression (GTEx, HPA), pathways, protein interactions (STRING), variant landscape... |
| tooluniverse-pharmacokinetics | Pharmacokinetic (PK) analysis of concentration-time data: non-compartmental analysis (NCA) for Cmax, Tmax, AUC (0-t and 0-infinity),... |
| tooluniverse-pharmacogenomics | Pharmacogenomics (PGx) research: drug-gene interactions (CPIC, PharmGKB), CPIC dosing guidelines, variant-drug-response... |
| tooluniverse-antibody-engineering | Therapeutic antibody engineering and optimization, lead-to-clinical-candidate |
| tooluniverse-toxicology | Drug and chemical toxicity assessment via adverse outcome pathways (AOPs), real-world FAERS adverse event signals, FDA labels, and... |
| tooluniverse-adverse-outcome-pathway | Map environmental and industrial chemicals to adverse outcome pathways (AOPs): molecular initiating event to organ-level toxicity |
| **Clinical, translational and safety** | |
| tooluniverse-clinical-data-integration | End-to-end drug safety review integrating FDA labels, FAERS adverse event reports, PRR/ROR disproportionality, pharmacogenomic... |
| tooluniverse-clinical-guidelines | Search and retrieve clinical practice guidelines from 12+ authoritative sources: NICE, WHO, NCCN, AHA, ADA, SIGN, USPSTF, IDSA,... |
| tooluniverse-clinical-risk-scoring | Compute and interpret validated bedside clinical risk scores and pretest probabilities for an INDIVIDUAL patient: pick the right... |
| tooluniverse-clinical-trial-design | Strategic clinical trial design feasibility assessment |
| tooluniverse-clinical-trial-matching | AI-driven patient-to-trial matching for precision oncology and rare-disease care |
| tooluniverse-precision-medicine-stratification | Patient stratification for precision medicine: integrate genomic, clinical, and therapeutic data to split patients into... |
| tooluniverse-precision-oncology | Cancer treatment recommendations from molecular profile (mutations + cancer type + biomarkers): FDA-approved + investigational... |
| tooluniverse-cancer-classification | Translate free-text tumor descriptions to OncoTree codes and resolve cancer subtypes/tissue hierarchy |
| tooluniverse-cell-line-profiling | Cancer cell-line selection and profiling for experimental model choice |
| tooluniverse-rare-disease-diagnosis | Rare disease differential diagnosis from patient phenotype: HPO term matching to candidate diseases (Orphanet, OMIM), gene panel... |

| Skill | Description |
|---|---|
| tooluniverse-diagnostic-test-evaluation | Diagnostic test / biomarker accuracy: sensitivity, specificity, PPV, NPV, likelihood ratios, accuracy from a 2x2 table; ROC curve,... |
| tooluniverse-pharmacovigilance | Drug safety and adverse event analysis: FAERS spontaneous-report mining, FDA black-box warnings, signal detection (PRR, ROR, IC),... |
| tooluniverse-adverse-event-detection | Detect and analyze adverse drug event signals using FDA FAERS reports, drug labels, and disproportionality statistics (PRR, ROR, IC) |
| tooluniverse-product-safety-surveillance | - |
| tooluniverse-immunotherapy-response-prediction | Predict patient response to immune checkpoint inhibitors (ICIs) by integrating tumor mutational burden (TMB), microsatellite... |
| **Immunology and infectious disease** | |
| tooluniverse-immunology | Immunology research workflows: antibody-antigen interactions, T/B cell repertoire, MHC/HLA binding prediction, autoimmune disease... |
| tooluniverse-immune-repertoire-analysis | TCR/BCR repertoire analysis: V(D)J segment usage, CDR3 sequence diversity, clonality scoring, antigen specificity matching to... |
| tooluniverse-hla-immunogenomics | HLA gene-family analysis and MHC-peptide binding for transplant compatibility, vaccine epitope coverage, and cancer immunotherapy |
| tooluniverse-vaccine-design | Computational vaccine candidate design: peptide/subunit vaccines via MHC-I/MHC-II epitope prediction (IEDB), population HLA... |
| tooluniverse-infectious-disease | Rapid pathogen characterization and drug repurposing for outbreaks |
| tooluniverse-microbiome-research | Microbiome research using MGnify, GTDB, ENA, OLS (ENVO biomes), and EuropePMC |
| tooluniverse-metagenomics-analysis | Microbiome and metagenomics analysis using MGnify, GTDB taxonomy, ENA sequencing data, and EuropePMC literature |
| **Multi-omics, systems and disease biology** | |
| tooluniverse-multi-omics-integration | Multi-omics integration: orchestrate per-layer analysis (transcriptomics, proteomics, epigenomics, genomics, metabolomics) then... |
| tooluniverse-multiomic-disease-characterization | Comprehensive disease characterization across genomics, transcriptomics, proteomics, and pathways for systems-level understanding |
| tooluniverse-spatial-omics-analysis | Spatial multi-omics interpretation pipeline |
| tooluniverse-systems-biology | Systems biology and pathway analysis integrating Reactome, KEGG, WikiPathways, BioCarta, NCI-Nature Pathway Interaction Database |

| Skill | Description |
|---|---|
| tooluniverse-disease-research | Generate comprehensive disease research reports covering genetics (causal genes, GWAS, OMIM), pathways (Reactome, KEGG), drugs... |
| tooluniverse-kegg-disease-drug | KEGG-based disease-drug-variant network research |
| tooluniverse-data-integration-analysis | Integrate computed statistical results (DEGs, GWAS hits, associations) with biological context from ToolUniverse databases... |
| tooluniverse-stem-cell-organoid | Stem cell, iPSC, and organoid research: pluripotency markers, differentiation protocol pathways, lineage commitment factors,... |
| tooluniverse-aging-senescence | Aging biology, cellular senescence, and longevity research |
| tooluniverse-neuroscience | Neuroscience research workflows: neuroanatomy, neural circuits, neurotransmitter biology, neurological/psychiatric disease... |
| **Cross-cutting methods, data and literature** | |
| tooluniverse-literature-deep-research | Deep literature review: PubMed, EuropePMC, bioRxiv preprints, citation networks, evidence synthesis |
| tooluniverse-meta-analysis | Meta-analysis / evidence synthesis: pool effect sizes across studies (odds ratios, risk ratios, hazard ratios, mean differences,... |
| tooluniverse-statistical-modeling | Statistical modeling: linear/logistic/ordinal/Poisson regression, ANOVA, Kruskal-Wallis, chi-square, Mann-Whitney, Cox survival,... |
| tooluniverse-epidemiological-analysis | End-to-end observational epidemiology analysis: from research question (PECO Population/Exposure/Comparator/Outcome) to... |
| tooluniverse-dataset-discovery | Find and evaluate research datasets for any scientific question |
| tooluniverse-data-wrangling | Universal data access patterns for downloading and parsing scientific data when ToolUniverse tools don't cover the source, only... |
| tooluniverse-image-analysis | Microscopy and quantitative imaging analysis: colony morphometry, fluorescence intensity quantification, cell-count statistics,... |
| tooluniverse-phylogenetics | Phylogenetic analysis: de novo multiple sequence alignment (Clustal Omega/MUSCLE/MAFFT via EBI_msa_align) and... |
| tooluniverse-ecology-biodiversity | Ecology, biodiversity, and conservation biology research: species identification (GBIF, NCBI Taxonomy), invasive species impact,... |

**Table S4:** The 130 research skills in TOOLUNIVERSE, grouped by application and data-modality area (as of 7 July 2026). Each skill is a natural-language-invokable workflow that orchestrates multiple tools into an end-to-end analysis; the skill catalogue is continually expanded.